

AI Assisted Coding

Lab Assignment 14

Web Frontend Development - AI-Assisted HTML/CSS/JS Generation

Name: BITLA SAIRAJ

Roll No: 2503a52162

Program: B. Tech

Year/Sem: II/I

Batch : 06

Course Title: AI Assisted Coding

Assignment Number: 14 / 24

Task 1: Responsive Homepage Layout for a Startup Website

Aim: To create a homepage for a startup with a header (navigation links), a main content section and a footer, and to use AI to generate CSS media queries so that the page adjusts properly on mobile, tablet and desktop screens.

Prompt 1:

Act as a senior front-end developer. Create a responsive homepage for a fictional startup called BrightNest using semantic HTML5 and plain CSS (no frameworks). HTML: a header with a logo and navigation links, a main section with a hero part, four feature cards and an about part, and a footer with company info, quick links and contact details. CSS: write it mobile-first with media queries for tablet (768px and up) and desktop (1024px and up). Use CSS variables, flexbox for the header and grid for the cards and footer. There should be no horizontal scrolling at 320px width, and the page should be accessible (skip link, visible focus styles, aria-label on the nav). Keep the code in two files, index.html and style.css, with short comments. Also explain which breakpoints you chose and why.

Prompt 2:

Review the CSS you produced for responsiveness problems at 320px, 375px, 768px, 1024px and 1440px. List any issue and fix it without adding a framework.

Code:

task1/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="BrightNest - simple workspace tools for early-stage startups.">
  <title>BrightNest - Workspace Tools for Startups</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <a class="skip-link" href="#main">Skip to main content</a>

  <!-- ===== Header with navigation ===== -->
  <header class="site-header">
    <div class="container header-inner">
      <a class="logo" href="#home">BrightNest</a>
      <nav aria-label="Main navigation">
        <ul class="nav-list">
          <li><a href="#home">Home</a></li>
          <li><a href="#features">Features</a></li>
          <li><a href="#about">About</a></li>
          <li><a href="#contact">Contact</a></li>
        </ul>
      </nav>
    </div>
  </header>

  <!-- ===== Main content ===== -->
  <main id="main">
    <section class="hero" id="home">
      <div class="container">
        <h1>Build your startup faster with BrightNest</h1>
        <p>Plan, track and ship in one simple workspace made for small teams.</p>
        <a class="btn" href="#contact">Get Started</a>
      </div>
    </section>

    <section class="features container" id="features">
      <h2>Why teams choose BrightNest</h2>
      <div class="card-grid">
        <article class="card">
          <span class="card-icon" aria-hidden="true">01</span>
          <h3>Plan</h3>
          <p>Turn ideas into clear roadmaps your whole team can follow.</p>
        </article>
        <article class="card">
          <span class="card-icon" aria-hidden="true">02</span>
          <h3>Collaborate</h3>
          <p>Share updates and feedback without long email threads.</p>
        </article>
        <article class="card">
          <span class="card-icon" aria-hidden="true">03</span>
          <h3>Track</h3>
          <p>See progress at a glance with simple, honest dashboards.</p>
        </article>
        <article class="card">
          <span class="card-icon" aria-hidden="true">04</span>
          <h3>Launch</h3>
          <p>Go from prototype to production with a checklist you trust.</p>
        </article>
      </div>
    </section>

    <section class="about" id="about">
      <div class="container about-inner">
        <div>
```

```

    <h2>About BrightNest</h2>
    <p>We are a small team that believes good tools should stay out of the way. BrightNest keeps planning,
communication and delivery in one place.</p>
</div>
<ul class="stats">
  <li><strong>500+</strong><span>Teams</span></li>
  <li><strong>12</strong><span>Countries</span></li>
  <li><strong>99.9%</strong><span>Uptime</span></li>
</ul>
</div>
</section>
</main>

<!-- ===== Footer ===== -->
<footer class="site-footer" id="contact">
  <div class="container footer-grid">
    <div>
      <h2>BrightNest</h2>
      <p>Workspace tools for early-stage startups.</p>
    </div>
    <div>
      <h3>Quick links</h3>
      <ul>
        <li><a href="#features">Features</a></li>
        <li><a href="#about">About</a></li>
      </ul>
    </div>
    <div>
      <h3>Contact</h3>
      <p>hello@brightnest.example<br>Hyderabad, India</p>
    </div>
  </div>
  <p class="footer-bottom container">&copy; 2026 BrightNest. All rights reserved.</p>
</footer>
</body>
</html>

```

task1/style.css

```

/* ===== Design tokens ===== */
:root {
  --brand: #2f5bea;
  --brand-dark: #1d3fb5;
  --ink: #1b2333;
  --muted: #5b667a;
  --soft: #f3f6fd;
  --radius: 12px;
  --max-width: 1100px;
}

/* ===== Base (mobile-first) ===== */
*, *::before, *::after { box-sizing: border-box; }
body {
  margin: 0;
  font-family: "Segoe UI", system-ui, -apple-system, Arial, sans-serif;
  color: var(--ink);
  line-height: 1.6;
}
h1, h2, h3 { line-height: 1.25; }
a { color: inherit; }
.container { width: min(100% - 2rem, var(--max-width)); margin-inline: auto; }

.skip-link { position: absolute; left: -999px; top: 0.5rem; background: #fff; padding: 0.5rem 1rem; }
.skip-link:focus { left: 0.5rem; z-index: 20; }
a:focus-visible, .btn:focus-visible { outline: 3px solid #ffb703; outline-offset: 2px; }

```

```

/* ===== Header ===== */
.site-header { position: sticky; top: 0; z-index: 10; background: #fff; border-bottom: 1px solid #e3e8f4; }
.header-inner { display: flex; flex-direction: column; align-items: center; gap: 0.25rem; padding: 0.75rem 0; }
.logo { font-size: 1.5rem; font-weight: 700; color: var(--brand); text-decoration: none; }
.nav-list { display: flex; flex-wrap: wrap; justify-content: center; gap: 0 1.25rem; margin: 0; padding: 0; list-style: none; }
.nav-list a { display: block; padding: 0.4rem 0; text-decoration: none; color: var(--muted); font-weight: 600; }
.nav-list a:hover { color: var(--brand); }

/* ===== Hero ===== */
.hero { padding: 3rem 0; text-align: center; color: #fff; background: linear-gradient(135deg, var(--brand), #6a4bd8); }
.hero h1 { margin: 0 0 1rem; font-size: clamp(1.8rem, 6vw, 3rem); }
.hero p { margin: 0 auto 1.5rem; max-width: 36rem; font-size: 1.1rem; }
.btn { display: inline-block; padding: 0.8rem 1.8rem; border-radius: 999px; background: #fff; color: var(--brand-dark); font-weight: 700; text-decoration: none; }
.btn:hover { background: #e8eeff; }

/* ===== Features ===== */
.features { padding: 3rem 0; }
.features h2 { margin-top: 0; text-align: center; }
.card-grid { display: grid; grid-template-columns: 1fr; gap: 1.25rem; }
.card { padding: 1.5rem; border: 1px solid #e3e8f4; border-radius: var(--radius); box-shadow: 0 2px 8px rgba(20, 40, 90, 0.06); }
.card h3 { margin: 0.75rem 0 0.25rem; }
.card p { margin: 0; color: var(--muted); }
.card-icon { display: inline-grid; place-items: center; width: 2.75rem; height: 2.75rem; border-radius: 50%; background: var(--soft); color: var(--brand); font-weight: 700; }

/* ===== About ===== */
.about { padding: 3rem 0; background: var(--soft); }
.about-inner { display: grid; gap: 1.5rem; }
.about h2 { margin-top: 0; }
.stats { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; margin: 0; padding: 0; list-style: none; text-align: center; }
.stats strong { display: block; font-size: 1.6rem; color: var(--brand); }
.stats span { color: var(--muted); }

/* ===== Footer ===== */
.site-footer { padding: 2.5rem 0 1rem; background: #121a2e; color: #cfd6e6; }
.site-footer h2, .site-footer h3 { margin-top: 0; color: #fff; font-size: 1.1rem; }
.site-footer ul { margin: 0; padding: 0; list-style: none; }
.footer-grid { display: grid; grid-template-columns: 1fr; gap: 1.5rem; }
.footer-bottom { margin: 2rem auto 0; padding-top: 1rem; border-top: 1px solid #2a3552; font-size: 0.9rem; text-align: center; }

/* ===== Tablet: 768px and up ===== */
@media (min-width: 768px) {
  .header-inner { flex-direction: row; justify-content: space-between; }
  .hero { padding: 4.5rem 0; }
  .card-grid { grid-template-columns: repeat(2, 1fr); }
  .footer-grid { grid-template-columns: repeat(2, 1fr); }
}

/* ===== Desktop: 1024px and up ===== */
@media (min-width: 1024px) {
  .hero { padding: 6rem 0; }
  .card-grid { grid-template-columns: repeat(4, 1fr); }
  .about-inner { grid-template-columns: 1.2fr 1fr; align-items: center; }
  .footer-grid { grid-template-columns: 2fr 1fr 1fr; }
}

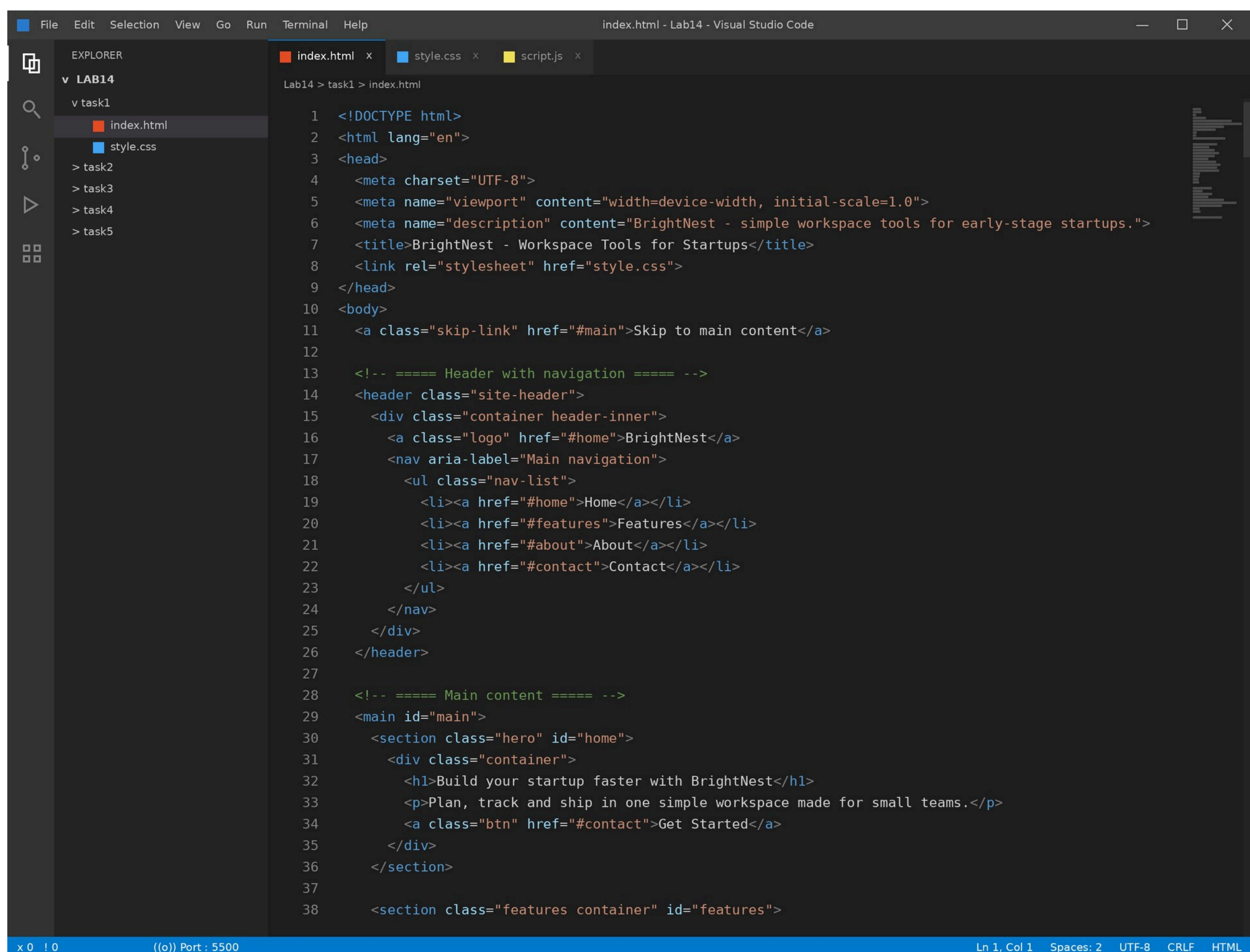
```

Explanation:

In index.html I used the semantic tags header, nav, main, section and footer, so the header, the content and the footer are clearly separated. The viewport meta tag is important because without it phones show the page at a fixed wide size and the media queries do not work properly. I also added a skip link and an aria-label for the navigation. The four feature cards are all written in the same way as article elements, so another card can be added without changing the CSS.

style.css is written mobile-first. The normal rules are for the phone layout (one column and a stacked header) and the media queries only add changes for bigger screens. At 768px the header becomes a row and the cards and the footer change to two columns. At 1024px the cards change to four columns and the footer to three columns. I used CSS variables for the colours, flexbox for the header and grid for the cards and footer. The container class uses width: min(100% - 2rem, 1100px), which leaves a small gap on phones and keeps the content centred on large screens. The heading size uses clamp(), so it grows smoothly instead of jumping at a breakpoint.

Code in VS Code:



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <meta name="description" content="BrightNest - simple workspace tools for early-stage startups.">
7   <title>BrightNest - Workspace Tools for Startups</title>
8   <link rel="stylesheet" href="style.css">
9 </head>
10 <body>
11   <a class="skip-link" href="#main">Skip to main content</a>
12
13   <!-- ===== Header with navigation ===== -->
14   <header class="site-header">
15     <div class="container header-inner">
16       <a class="logo" href="#home">BrightNest</a>
17       <nav aria-label="Main navigation">
18         <ul class="nav-list">
19           <li><a href="#home">Home</a></li>
20           <li><a href="#features">Features</a></li>
21           <li><a href="#about">About</a></li>
22           <li><a href="#contact">Contact</a></li>
23         </ul>
24       </nav>
25     </div>
26   </header>
27
28   <!-- ===== Main content ===== -->
29   <main id="main">
30     <section class="hero" id="home">
31       <div class="container">
32         <h1>Build your startup faster with BrightNest</h1>
33         <p>Plan, track and ship in one simple workspace made for small teams.</p>
34         <a class="btn" href="#contact">Get Started</a>
35       </div>
36     </section>
37
38     <section class="features container" id="features">
```

```
1  /* ===== Design tokens ===== */
2  :root {
3    --brand: #2f5bea;
4    --brand-dark: #1d3fb5;
5    --ink: #1b2333;
6    --muted: #5b667a;
7    --soft: #f3f6fd;
8    --radius: 12px;
9    --max-width: 1100px;
10 }
11
12 /* ===== Base (mobile-first) ===== */
13 *, *::before, *::after { box-sizing: border-box; }
14 body {
15   margin: 0;
16   font-family: "Segoe UI", system-ui, -apple-system, Arial, sans-serif;
17   color: var(--ink);
18   line-height: 1.6;
19 }
20 h1, h2, h3 { line-height: 1.25; }
21 a { color: inherit; }
22 .container { width: min(100% - 2rem, var(--max-width)); margin-inline: auto; }
23
24 .skip-link { position: absolute; left: -999px; top: 0.5rem; background: #fff; padding: 0.5rem 1rem; }
25 .skip-link:focus { left: 0.5rem; z-index: 20; }
26 a:focus-visible, .btn:focus-visible { outline: 3px solid #ffb703; outline-offset: 2px; }
27
28 /* ===== Header ===== */
29 .site-header { position: sticky; top: 0; z-index: 10; background: #fff; border-bottom: 1px solid #e3e3e3; }
30 .header-inner { display: flex; flex-direction: column; align-items: center; gap: 0.25rem; padding: 0 1rem; }
31 .logo { font-size: 1.5rem; font-weight: 700; color: var(--brand); text-decoration: none; }
32 .nav-list { display: flex; flex-wrap: wrap; justify-content: center; gap: 0 1.25rem; margin: 0; padding: 0 1rem; }
33 .nav-list a { display: block; padding: 0.4rem 0; text-decoration: none; color: var(--muted); font-weight: 500; }
34 .nav-list a:hover { color: var(--brand); }
35
36 /* ===== Hero ===== */
37 .hero { padding: 3rem 0; text-align: center; color: #fff; background: linear-gradient(135deg, var(--brand), var(--brand-dark)); }
38 .hero h1 { margin: 0 0 1rem; font-size: clamp(1.8rem, 6vw, 3rem); }
```

Output:

I opened the page with Live Server and took the output at three screen widths. The header, main content and footer are separate at every size.

Build your startup faster with BrightNest

Plan, track and ship in one simple workspace made for small teams.

Get Started

Why teams choose BrightNest

01

Plan

Turn ideas into clear roadmaps your whole team can follow.

02

Collaborate

Share updates and feedback without long email threads.

03

Track

See progress at a glance with simple, honest dashboards.

04

Launch

Go from prototype to production with a checklist you trust.

About BrightNest

We are a small team that believes good tools should stay out of the way. BrightNest keeps planning, communication and delivery in one place.

500+
Teams

12
Countries

99.9%
Uptime

BrightNest

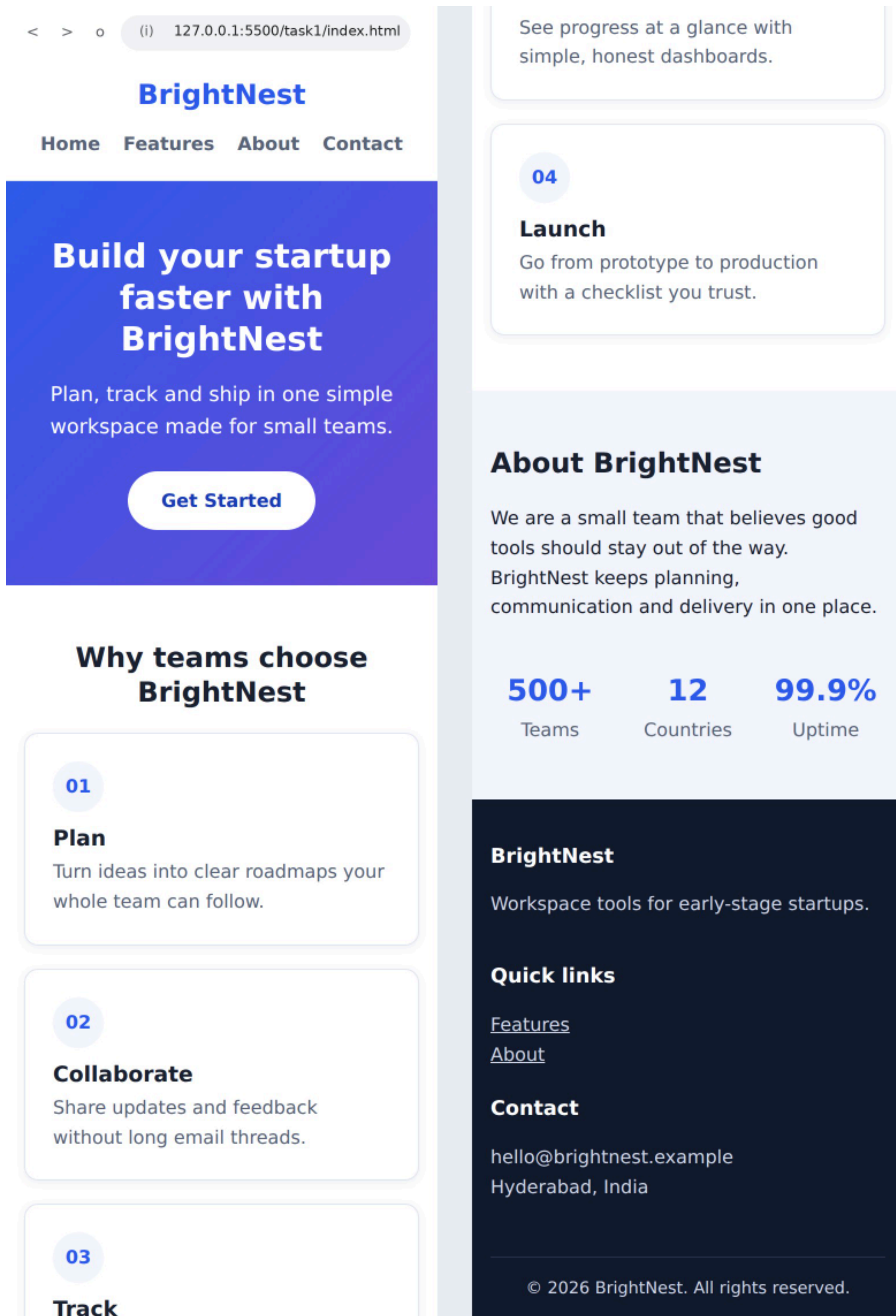
Workspace tools for early-stage startups.

Quick links

[Features](#)
[About](#)

Contact

hello@brightnest.example
Hyderabad, India



Testing:

To check the layout properly I also loaded the page in Chromium at 320, 375, 768, 1024 and 1440 pixels width with a small script. There was no horizontal scrolling at any width,

the cards were in 1, 1, 2, 4 and 4 columns, the header changed from stacked to a row at 768px, and the header, main and footer stayed in the correct order (6 of 6 checks passed).

Observations:

The breakpoints 768px and 1024px are common values and not a rule given in the lab. The navigation has only four links, so on a phone it just wraps to the next line; if more links were added a menu button would be needed. I have not tested the page on a real phone or in Firefox or Safari.

Task 2: Interactive Call-to-Action Button

Aim: To add a call-to-action button to a marketing page and use AI generated JavaScript to show a visible response when the button is clicked, with comments explaining the logic and without reloading the page.

Prompt 1:

Act as a front-end developer. Create a small marketing page in HTML and CSS with a heading, a short paragraph and one call-to-action button "Get Started Free". Write JavaScript in a separate script.js file, using addEventListener (no inline onclick and no libraries), that shows a visible message on the page when the button is clicked. The page must not reload. Make the message accessible for screen readers.

Prompt 2:

Add meaningful comments to the JavaScript that explain each step of the logic for a beginner, and keep the code readable. Do not change the behaviour.

Code:

task2/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Call-to-Action Button</title>
  <link rel="stylesheet" href="style.css">
  <!-- "defer" runs the script after the HTML is parsed, so the elements exist -->
  <script src="script.js" defer></script>
</head>
<body>
  <main class="promo">
    <h1>Launch your idea in 30 days</h1>
    <p>Join the BrightNest free trial and get your first project set up in minutes.</p>

    <!-- type="button" so the button never submits a form or reloads the page -->
    <button type="button" id="cta-button" class="cta">Get Started Free</button>
```

```

    <!-- aria-live announces the message to screen-reader users -->
    <p id="cta-message" class="cta-message" role="status" aria-live="polite" hidden></p>
  </main>
</body>
</html>

```

task2/style.css

```

body { margin: 0; min-height: 100vh; display: grid; place-items: center; padding: 1rem;
  font-family: "Segoe UI", system-ui, Arial, sans-serif; color: #1b2333; background: #f3f6fd; }
.promo { width: min(100%, 36rem); padding: 2.5rem 1.5rem; text-align: center; background: #fff;
  border-radius: 16px; box-shadow: 0 8px 30px rgba(20, 40, 90, 0.12); }
.promo h1 { margin: 0 0 0.75rem; font-size: clamp(1.6rem, 5vw, 2.2rem); }
.promo p { margin: 0 0 1.5rem; color: #5b667a; }
.cta { padding: 0.9rem 2.2rem; border: 0; border-radius: 999px; background: #2f5bea; color: #fff;
  font-size: 1.1rem; font-weight: 700; cursor: pointer; transition: background 0.2s, transform 0.1s; }
.cta:hover { background: #1d3fb5; }
.cta:active { transform: scale(0.97); }
.cta:focus-visible { outline: 3px solid #ffb703; outline-offset: 3px; }
.cta-message { margin: 1.5rem 0 0; padding: 0.9rem 1rem; border-radius: 10px; background: #e7f7ee;
  border: 1px solid #8fd3ab; color: #12633a; font-weight: 600; }
.cta-message[hidden] { display: none; }
@media (prefers-reduced-motion: reduce) { .cta { transition: none; } }

```

task2/script.js

```

// Grab the elements we need once, when the script starts (it is loaded with "defer").
const ctaButton = document.getElementById("cta-button");
const messageBox = document.getElementById("cta-message");

// Remember how many times the button was clicked. Because the page is never
// reloaded, this value keeps growing - which proves no reload happens.
let clickCount = 0;

// Run this function every time the button is clicked.
ctaButton.addEventListener("click", function () {
  clickCount += 1;

  // Choose a message: a friendly welcome first, a gentle reminder afterwards.
  if (clickCount === 1) {
    messageBox.textContent = "Thank you! Your free trial is being set up.";
  } else {
    messageBox.textContent = "Already on it! You have clicked " + clickCount + " times.";
  }

  // The message starts hidden; un-hide it so the user can see the response.
  messageBox.hidden = false;
});

```

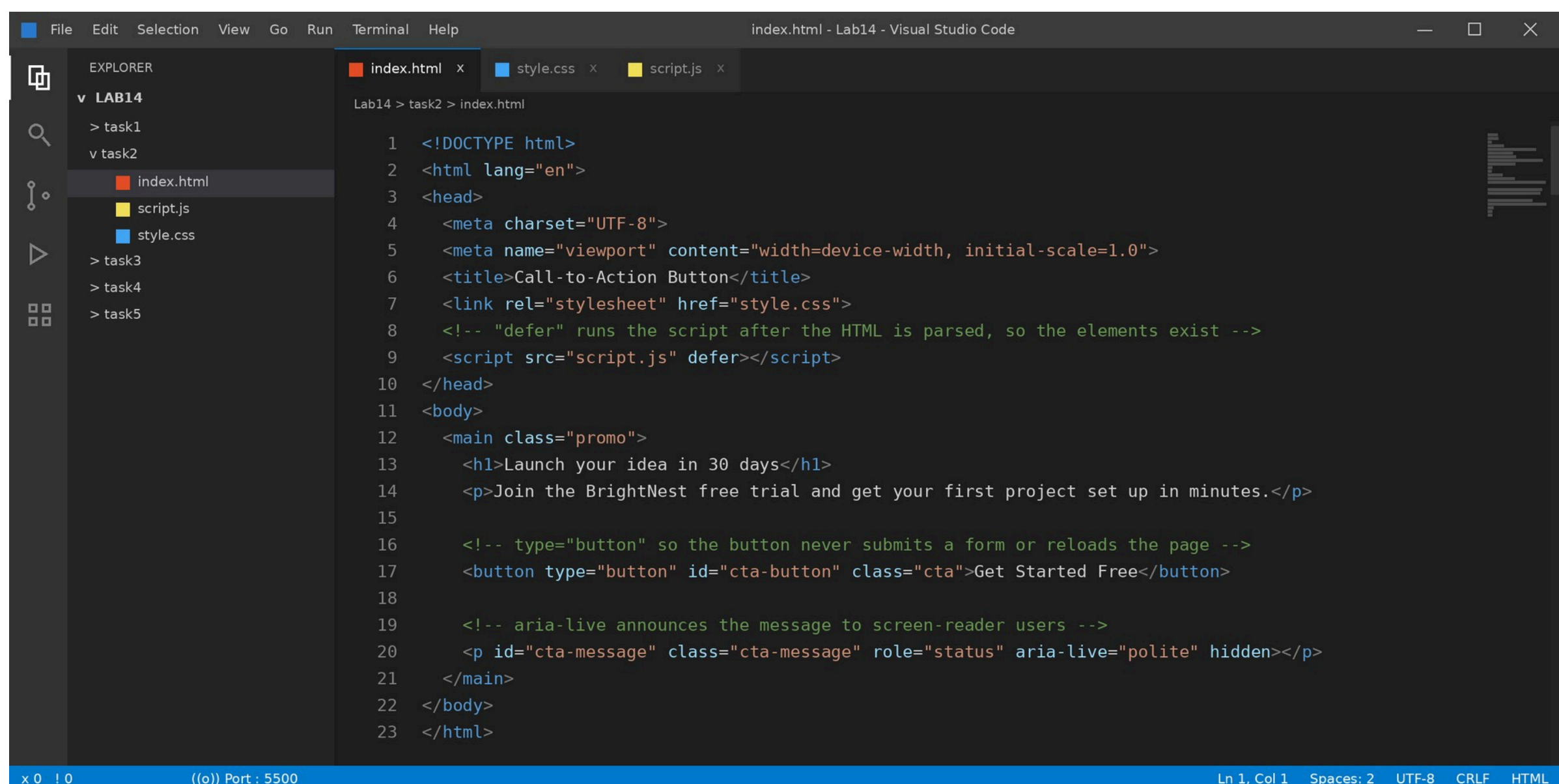
Explanation:

The button in index.html has type="button", so it can never submit a form or reload the page. The message paragraph is hidden at the start with the hidden attribute, and it has role="status" and aria-live so that screen readers read the message when it appears. The script is loaded with defer so it runs only after the HTML has loaded.

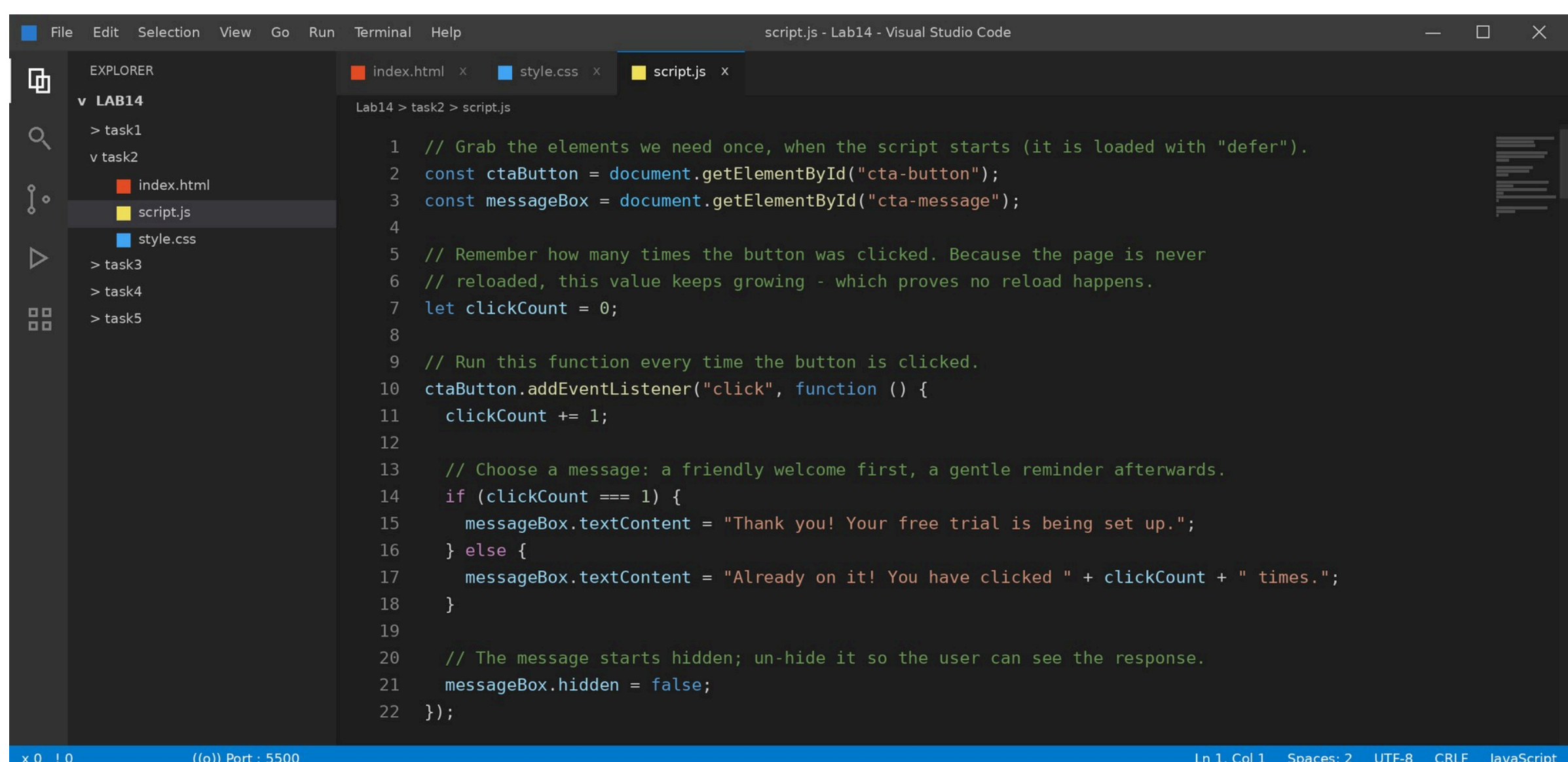
In script.js I first store the button and the message box in variables. The variable

clickCount remembers how many times the button was clicked. Then addEventListener runs a function on every click. The function increases the count and uses an if/else to show a welcome message on the first click and a different message with the count on the next clicks. At the end hidden is set to false so the message becomes visible. Since the count keeps increasing after every click, it shows that the page is not reloading.

Code in VS Code:



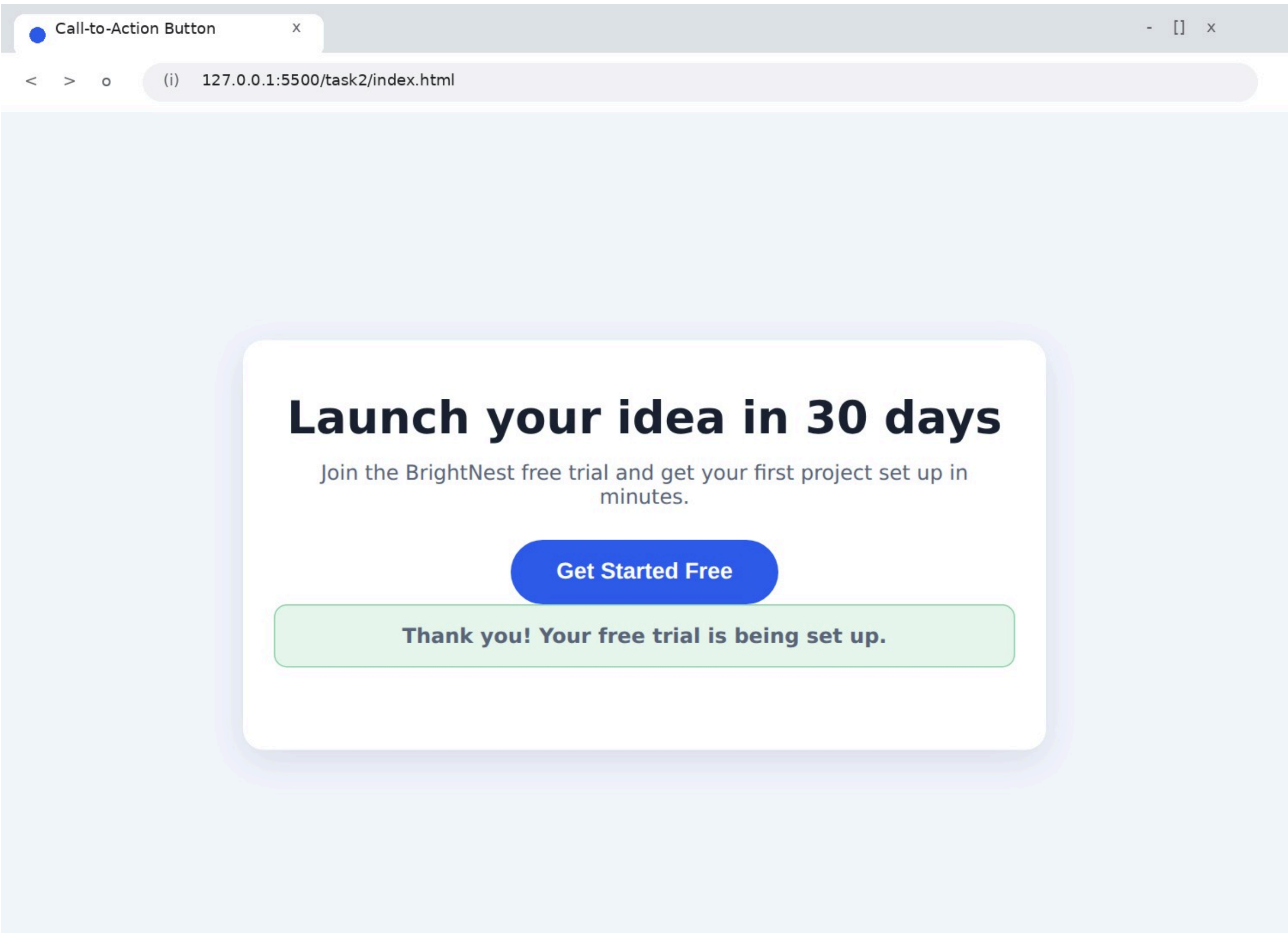
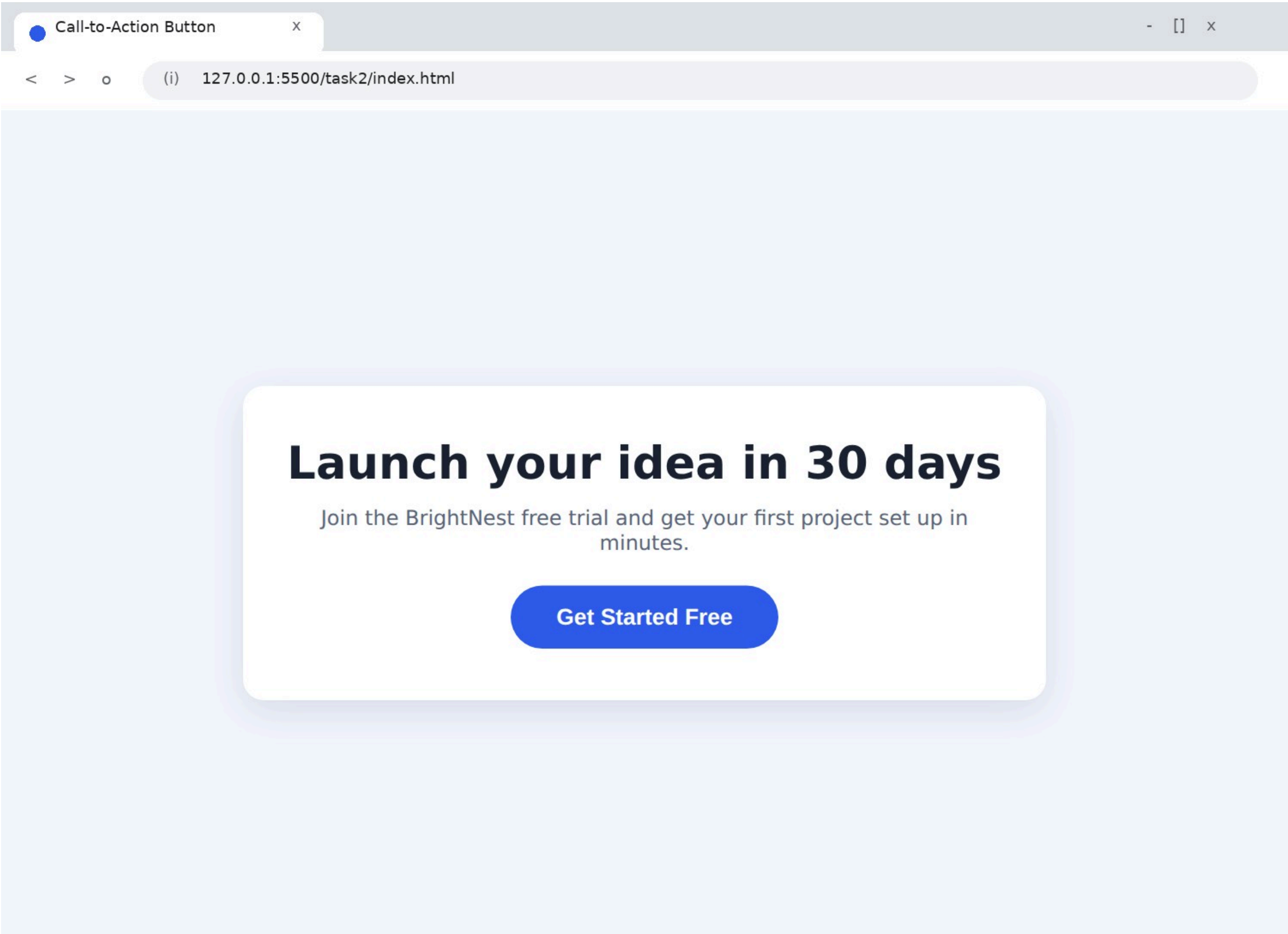
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Call-to-Action Button</title>
7   <link rel="stylesheet" href="style.css">
8   <!-- "defer" runs the script after the HTML is parsed, so the elements exist -->
9   <script src="script.js" defer></script>
10 </head>
11 <body>
12   <main class="promo">
13     <h1>Launch your idea in 30 days</h1>
14     <p>Join the BrightNest free trial and get your first project set up in minutes.</p>
15
16     <!-- type="button" so the button never submits a form or reloads the page -->
17     <button type="button" id="cta-button" class="cta">Get Started Free</button>
18
19     <!-- aria-live announces the message to screen-reader users -->
20     <p id="cta-message" class="cta-message" role="status" aria-live="polite" hidden></p>
21   </main>
22 </body>
23 </html>
```

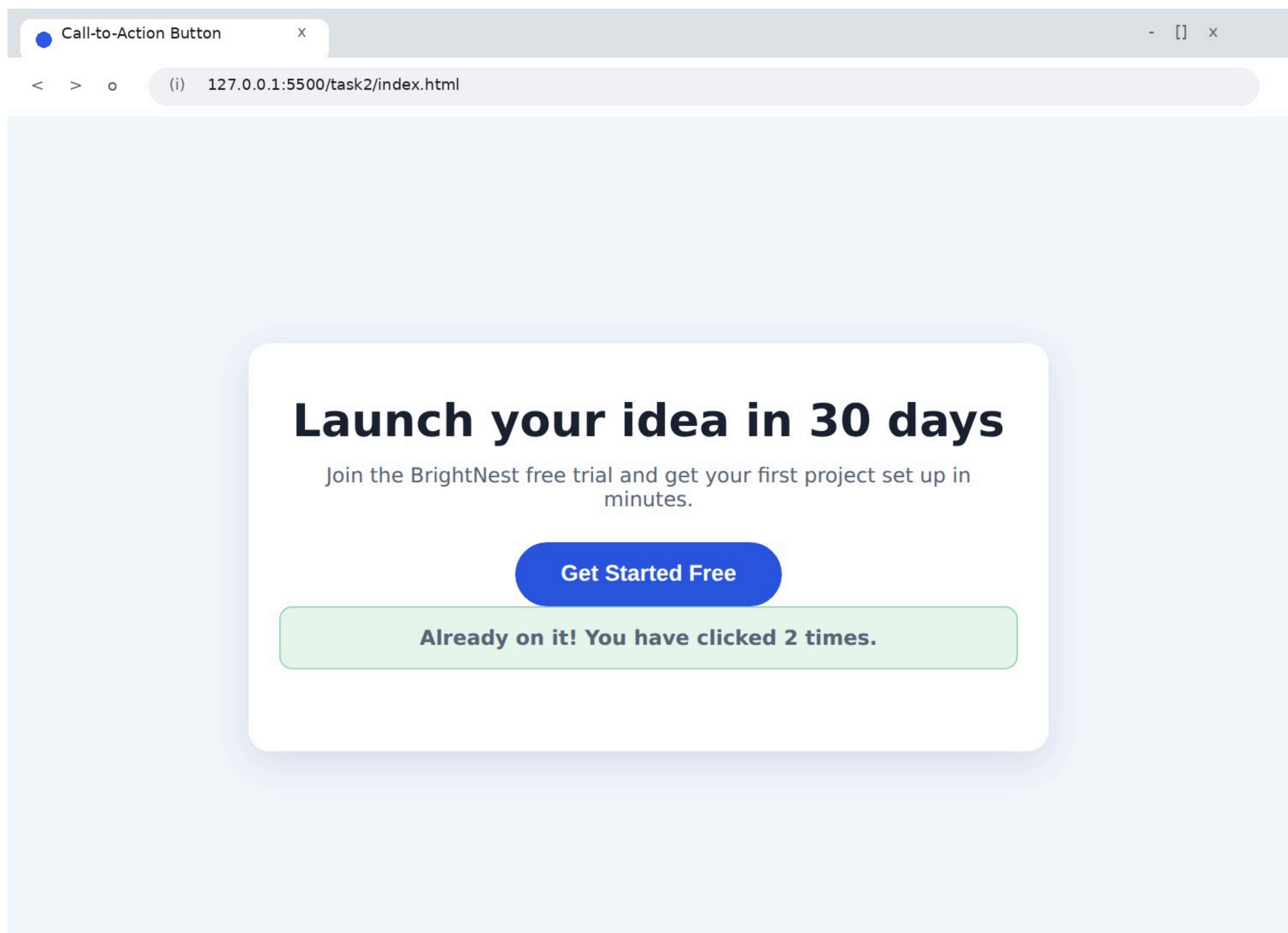


```
1 // Grab the elements we need once, when the script starts (it is loaded with "defer").
2 const ctaButton = document.getElementById("cta-button");
3 const messageBox = document.getElementById("cta-message");
4
5 // Remember how many times the button was clicked. Because the page is never
6 // reloaded, this value keeps growing - which proves no reload happens.
7 let clickCount = 0;
8
9 // Run this function every time the button is clicked.
10 ctaButton.addEventListener("click", function () {
11   clickCount += 1;
12
13   // Choose a message: a friendly welcome first, a gentle reminder afterwards.
14   if (clickCount === 1) {
15     messageBox.textContent = "Thank you! Your free trial is being set up.";
16   } else {
17     messageBox.textContent = "Already on it! You have clicked " + clickCount + " times.";
18   }
19
20   // The message starts hidden; un-hide it so the user can see the response.
21   messageBox.hidden = false;
22 });
```

Output:

The output is shown before the click, after the first click and after the second click.





Testing:

I tested it by clicking the button in Chromium using a script. The message was hidden before the click, appeared after the first click, and changed to show "2 times" after the second click. To confirm that the page did not reload, I saved a value in the page memory before clicking and it was still there after the clicks. There were no JavaScript errors (5 of 5 checks passed).

Observations:

I used an on-page message instead of `alert()` because `alert()` blocks the whole page and cannot be styled. If an alert is required, the two `textContent` lines in `script.js` can be replaced with `alert("Thank you! Your free trial is being set up.")` and everything else stays the same. The click count is not saved, so it starts from zero again when the page is refreshed.

Task 3: Contact Form with Client-Side Validation

Aim: To design a contact form with Name, Email and Message fields and use AI generated JavaScript to stop empty submissions, check the email format, show inline

error messages for wrong input and show a success message for correct input.

Prompt 1:

Act as a front-end developer. Build a contact form with Name, Email and Message fields in HTML and CSS and write client-side validation in a separate script.js. Rules: no empty submissions (a value with only spaces should also count as empty), a valid email format check, name of at least 2 characters and message of at least 10 characters. Show a clear error message under each wrong field and a success message when everything is correct, without reloading the page. Use novalidate on the form so the custom messages are shown, labels linked to the inputs, and aria-invalid and aria-describedby for accessibility. Explain the email regular expression.

Prompt 2:

Review the validation for edge cases (a name with only spaces, "a@b", "a b@c.com", "@c.com", "a@@c.com") and accessibility problems, and list the limitations of client-side-only validation.

Code:

task3/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact Us</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main class="card">
    <h1>Contact Us</h1>
    <p class="intro">Send us your query and we will reply within two working days.</p>

    <!-- "novalidate" turns off the browser's own pop-ups so our JavaScript messages are shown -->
    <form id="contact-form" novalidate>
      <div class="field">
        <label for="name">Name</label>
        <input type="text" id="name" name="name" autocomplete="name" aria-describedby="name-error">
        <p class="error" id="name-error"></p>
      </div>

      <div class="field">
        <label for="email">Email</label>
        <input type="email" id="email" name="email" autocomplete="email" aria-describedby="email-error">
        <p class="error" id="email-error"></p>
      </div>

      <div class="field">
        <label for="message">Message</label>
        <textarea id="message" name="message" rows="4" aria-describedby="message-error"></textarea>
        <p class="error" id="message-error"></p>
      </div>
```

```

        <button type="submit" class="submit">Send Message</button>
        <p id="form-success" class="success" role="status" hidden></p>
    </form>
</main>
</body>
</html>

```

task3/style.css

```

body { margin: 0; min-height: 100vh; display: grid; place-items: center; padding: 1rem;
    font-family: "Segoe UI", system-ui, Arial, sans-serif; color: #1b2333; background: #f3f6fd; }
.card { width: min(100%, 32rem); padding: 2rem 1.5rem; background: #fff; border-radius: 16px;
    box-shadow: 0 8px 30px rgba(20, 40, 90, 0.12); }
h1 { margin: 0 0 0.25rem; }
.intro { margin: 0 0 1.5rem; color: #5b667a; }
.field { margin-bottom: 1rem; }
label { display: block; margin-bottom: 0.3rem; font-weight: 600; }
input, textarea { width: 100%; padding: 0.7rem 0.8rem; border: 2px solid #c9d2e6; border-radius: 8px;
    font: inherit; }
input:focus, textarea:focus { outline: 3px solid rgba(47, 91, 234, 0.35); border-color: #2f5bea; }
/* Red border when JavaScript marks a field invalid; green when it is valid */
input[aria-invalid="true"], textarea[aria-invalid="true"] { border-color: #d62839; }
input[aria-invalid="false"], textarea[aria-invalid="false"] { border-color: #2e9e5b; }
.error { min-height: 1.2rem; margin: 0.3rem 0 0; color: #d62839; font-size: 0.9rem; }
.submit { padding: 0.8rem 1.6rem; border: 0; border-radius: 8px; background: #2f5bea; color: #fff;
    font: inherit; font-weight: 700; cursor: pointer; }
.submit:hover { background: #1d3fb5; }
.success { margin: 1rem 0 0; padding: 0.9rem 1rem; border: 1px solid #8fd3ab; border-radius: 10px;
    background: #e7f7ee; color: #12633a; font-weight: 600; }
.success[hidden] { display: none; }

```

task3/script.js

```

const form = document.getElementById("contact-form");
const successBox = document.getElementById("form-success");

// Simple, practical email pattern: text@text.domain (no spaces).
// It is NOT a complete RFC check - a real site must also validate on the server.
const EMAIL_PATTERN = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;

// One validator per field. Each returns an error message, or "" when the value is valid.
const validators = {
    name(value) {
        if (value === "") return "Name is required.";
        if (value.length < 2) return "Name must be at least 2 characters.";
        return "";
    },
    email(value) {
        if (value === "") return "Email is required.";
        if (!EMAIL_PATTERN.test(value)) return "Please enter a valid email, e.g. name@example.com.";
        return "";
    },
    message(value) {
        if (value === "") return "Message is required.";
        if (value.length < 10) return "Message must be at least 10 characters.";
        return "";
    },
};

// Validate one field, show or clear its inline error, and return true if it is valid.
function validateField(field) {
    const errorText = validators[field.name](field.value.trim()); // trim() makes " " count as empty
    document.getElementById(field.id + "-error").textContent = errorText;
    field.setAttribute("aria-invalid", errorText === "" ? "false" : "true");
}

```

```

    return errorText === "";
}

// Check every field when the form is submitted.
form.addEventListener("submit", function (event) {
    event.preventDefault(); // stop the browser from sending the form / reloading the page
    successBox.hidden = true;

    let firstInvalid = null;
    for (const field of form.elements) {
        if (field.name && !validateField(field) && firstInvalid === null) {
            firstInvalid = field;
        }
    }

    if (firstInvalid) {
        firstInvalid.focus(); // move the keyboard focus to the first problem
        return;
    }

    // All fields are valid: show the success message and clear the form.
    successBox.textContent = "Thank you, " + form.elements.name.value.trim() + "! Your message has been sent.";
    successBox.hidden = false;
    form.reset();
    for (const field of form.elements) field.removeAttribute("aria-invalid");
});

// Give feedback as soon as the user leaves a field, and clear errors while they fix them.
form.addEventListener("focusout", function (event) {
    if (event.target.name) validateField(event.target);
});
form.addEventListener("input", function (event) {
    if (event.target.getAttribute("aria-invalid") === "true") validateField(event.target);
});

```

Explanation:

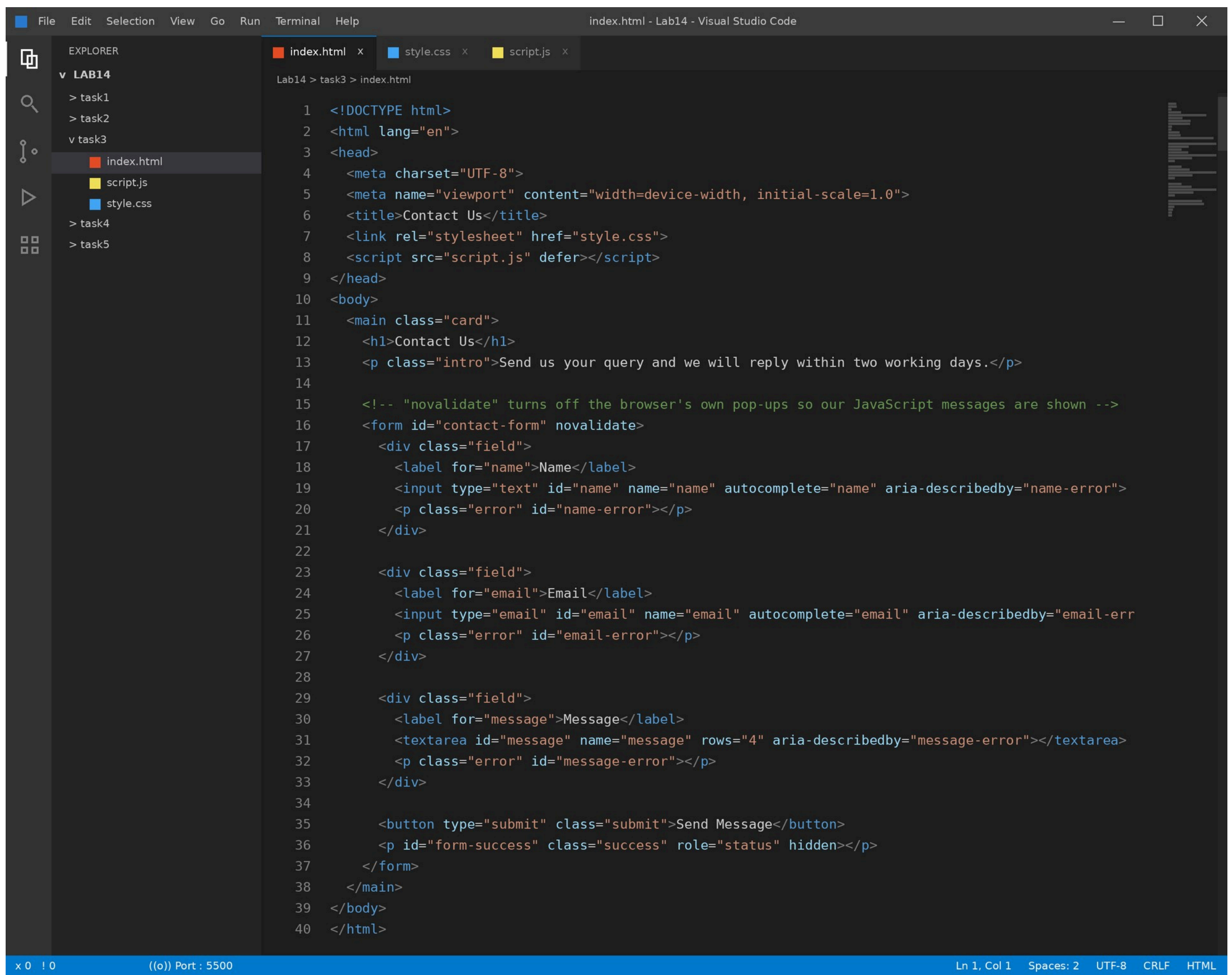
In index.html every input has a label connected with the for attribute, and below every input there is an empty paragraph for its error message, which is linked with aria-describedby. The form has novalidate, which switches off the default browser pop-up messages so that our own inline messages are shown.

In style.css a field with aria-invalid="true" gets a red border and a field with aria-invalid="false" gets a green border. The error text has a minimum height so that the form does not jump when a message appears.

In script.js there is a validators object with one small function for each field. Each function returns an error message, or an empty string if the value is correct. The email pattern `/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/` means some characters without spaces or @, then one @, then more characters, a dot and at least two more characters, for example `sairaj@example.com`. The function `validateField()` trims the value (so spaces only count as empty), writes the error message under the field and sets `aria-invalid`. On submit, `event.preventDefault()` stops the page from reloading, all the fields are checked and the

focus goes to the first wrong field. If all fields are correct, the success message is shown and the form is cleared. A field is also checked when the user leaves it, and its error disappears while the user is correcting it.

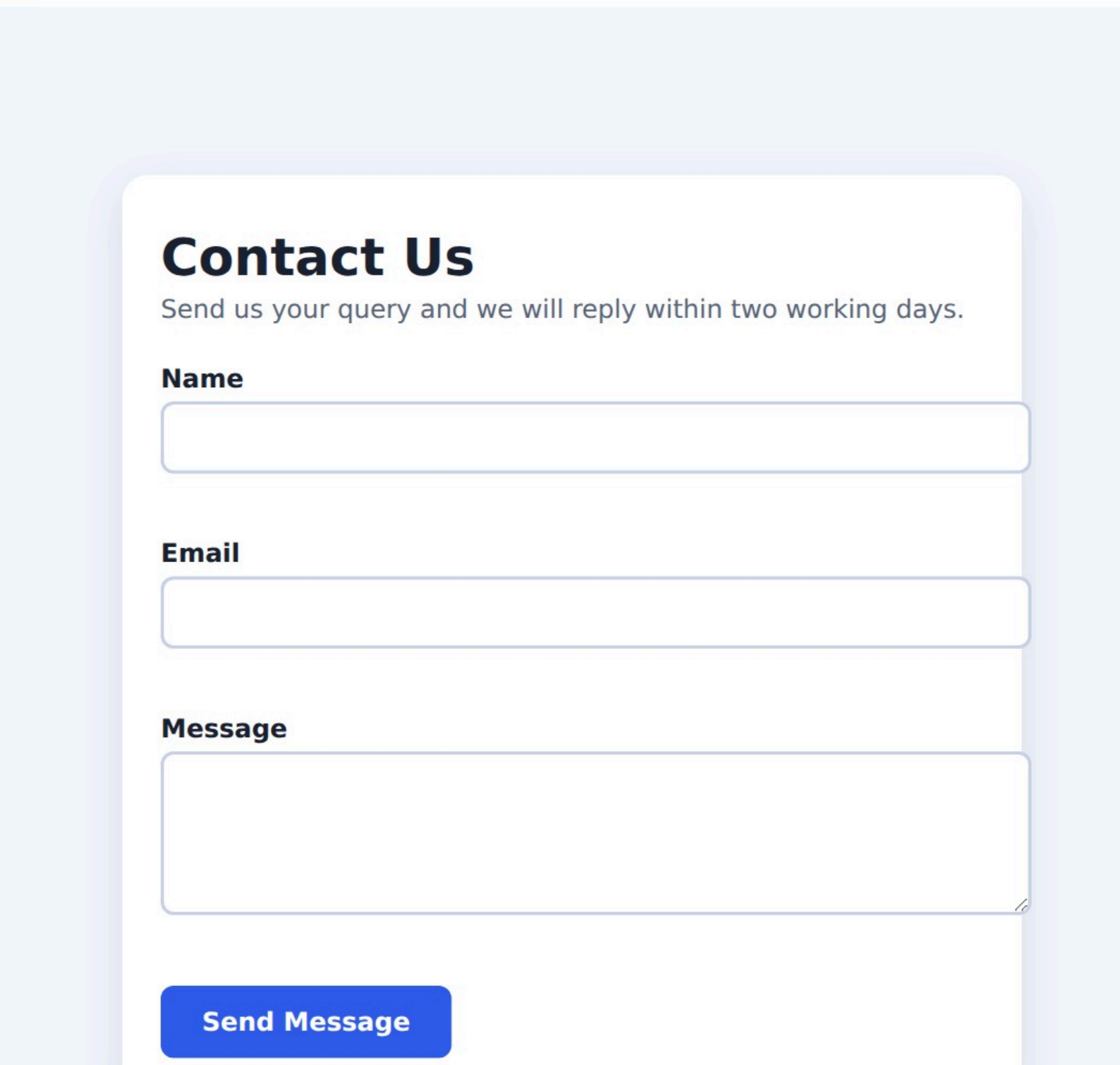
Code in VS Code:



```
1  const form = document.getElementById("contact-form");
2  const successBox = document.getElementById("form-success");
3
4  // Simple, practical email pattern: text@text.domain (no spaces).
5  // It is NOT a complete RFC check - a real site must also validate on the server.
6  const EMAIL_PATTERN = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
7
8  // One validator per field. Each returns an error message, or "" when the value is valid.
9  const validators = {
10   name(value) {
11     if (value === "") return "Name is required.";
12     if (value.length < 2) return "Name must be at least 2 characters.";
13     return "";
14   },
15   email(value) {
16     if (value === "") return "Email is required.";
17     if (!EMAIL_PATTERN.test(value)) return "Please enter a valid email, e.g. name@example.com.";
18     return "";
19   },
20   message(value) {
21     if (value === "") return "Message is required.";
22     if (value.length < 10) return "Message must be at least 10 characters.";
23     return "";
24   },
25 };
26
27 // Validate one field, show or clear its inline error, and return true if it is valid.
28 function validateField(field) {
29   const errorText = validators[field.name](field.value.trim()); // trim() makes " " count as empty
30   document.getElementById(field.id + "-error").textContent = errorText;
31   field.setAttribute("aria-invalid", errorText === "" ? "false" : "true");
32   return errorText === "";
33 }
34
35 // Check every field when the form is submitted.
36 form.addEventListener("submit", function (event) {
37   event.preventDefault(); // stop the browser from sending the form / reloading the page
38   successBox.hidden = true;
```

Output:

The output screenshots show the empty form, the form submitted empty, the form with an invalid email and short message, and the form with correct input.



Contact Us

Send us your query and we will reply within two working days.

Name

Email

Message

Send Message

Contact Us

X

- [] x

< > o (i) 127.0.0.1:5500/task3/index.html

Contact Us

Send us your query and we will reply within two working days.

Name

Name is required.

Email

Email is required.

Message

Message is required.

Send Message

Contact Us

X

- [] x

< > o (i) 127.0.0.1:5500/task3/index.html

Contact Us

Send us your query and we will reply within two working days.

Name

Sairaj

Email

sairaj@

Please enter a valid email, e.g. name@example.com.

Message

Hi

Message must be at least 10 characters.

Send Message

Testing:

I filled the form with correct and wrong values in Chromium using a script and checked the exact messages. An empty submit showed three "required" errors and moved the focus to the name field, a name with only spaces was treated as empty, "sairaj@" and a short message were rejected, and a@b, "a b@c.com", @c.com and a@@c.com were also rejected. A correct form showed the success message, cleared the fields and did not reload the page (12 of 12 checks passed).

Observations:

This validation only runs in the browser, so anyone can bypass it; a real website must check the same things again on the server. The form in this lab does not send the data anywhere, the success message is only shown after the validation. The email pattern is simple on purpose and it cannot confirm that an email address really exists.

Task 4: Dynamic List Management for a Product Page

Aim: To create a product list in HTML and use AI generated JavaScript to add a new item

when the Add button is clicked and remove the last item when the Remove button is clicked, updating the page without a reload.

Prompt 1:

Act as a front-end developer. Create a product page with an HTML ul list that starts with three products. Add an "Add" button that appends a new li and a "Remove" button that removes the last li. Use document.createElement and appendChild (do not use innerHTML with user input). Add an optional text box for the product name (the default name should be "Product N"), show how many items are in the list, and handle the empty list by disabling the Remove button and showing a message. Keep the JavaScript in script.js with small, well-named functions and comments.

Prompt 2:

Explain the code to a beginner, and tell me what happens if a user types HTML such as `<b>bold</b>` into the product name box.

Code:

task4/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Product List</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main class="panel">
    <h1>Our Products</h1>
    <p id="item-count" class="count" aria-live="polite"></p>

    <ul id="product-list" class="product-list">
      <li>Wireless Mouse</li>
      <li>Mechanical Keyboard</li>
      <li>USB-C Hub</li>
    </ul>
    <p id="list-message" class="note" role="status"></p>

    <div class="controls">
      <label for="item-name" class="visually-hidden">New product name (optional)</label>
      <input type="text" id="item-name" placeholder="New product name (optional)" maxlength="40">
      <button type="button" id="add-btn" class="btn add">Add</button>
      <button type="button" id="remove-btn" class="btn remove">Remove</button>
    </div>
  </main>
</body>
</html>
```

task4/style.css

```
body { margin: 0; min-height: 100vh; display: grid; place-items: center; padding: 1rem;
  font-family: "Segoe UI", system-ui, Arial, sans-serif; color: #1b2333; background: #f3f6fd; }
```

```

.panel { width: min(100%, 34rem); padding: 2rem 1.5rem; background: #fff; border-radius: 16px;
  box-shadow: 0 8px 30px rgba(20, 40, 90, 0.12); }
h1 { margin: 0 0 0.25rem; }
.count { margin: 0 0 1rem; color: #5b667a; font-weight: 600; }
.product-list { margin: 0 0 0.5rem; padding: 0; list-style: none; }
.product-list li { padding: 0.75rem 1rem; margin-bottom: 0.5rem; border: 1px solid #e3e8f4;
  border-left: 5px solid #2f5bea; border-radius: 8px; background: #fafcff; }
.note { min-height: 1.4rem; margin: 0 0 1rem; color: #b4540a; font-size: 0.95rem; }
.controls { display: flex; flex-wrap: wrap; gap: 0.5rem; }
.controls input { flex: 1 1 12rem; padding: 0.7rem 0.8rem; border: 2px solid #c9d2e6; border-radius: 8px; font: inherit; }
.controls input:focus { outline: 3px solid rgba(47, 91, 234, 0.35); border-color: #2f5bea; }
.btn { padding: 0.7rem 1.4rem; border: 0; border-radius: 8px; color: #fff; font: inherit; font-weight: 700; cursor: pointer; }
.add { background: #2e9e5b; }
.add:hover { background: #247c47; }
.remove { background: #d62839; }
.remove:hover { background: #a91e2c; }
.btn:disabled { background: #b8c0d0; cursor: not-allowed; }
.visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); }

```

task4/script.js

```

const productList = document.getElementById("product-list");
const itemNameInput = document.getElementById("item-name");
const addButton = document.getElementById("add-btn");
const removeButton = document.getElementById("remove-btn");
const countLabel = document.getElementById("item-count");
const messageLabel = document.getElementById("list-message");

```

```

// Used to give default names ("Product 4", "Product 5", ...) to new items.
let nextNumber = productList.children.length + 1;

```

```

// Refresh the "Items: N" label and disable Remove when the list is empty.
function updateStatus() {
  const total = productList.children.length;
  countLabel.textContent = "Items in list: " + total;
  removeButton.disabled = total === 0;
  if (total === 0) messageLabel.textContent = "The list is empty - add a product.";
}

```

```

// ADD: build a new <li> and append it to the end of the list.
addButton.addEventListener("click", function () {
  const typedName = itemNameInput.value.trim();
  const item = document.createElement("li");
  //.textContent (not innerHTML) so typed text can never be run as HTML
  item.textContent = typedName !== "" ? typedName : "Product " + nextNumber;
  productList.appendChild(item);

```

```

  nextNumber += 1;
  itemNameInput.value = "";
  messageLabel.textContent = "";
  updateStatus();
});

```

```

// REMOVE: delete the last <li>, if there is one.
removeButton.addEventListener("click", function () {
  const lastItem = productList.lastElementChild;
  if (lastItem) {
    lastItem.remove();
    messageLabel.textContent = "";
  }
  updateStatus();
});

```

```

updateStatus(); // show the correct count when the page first loads

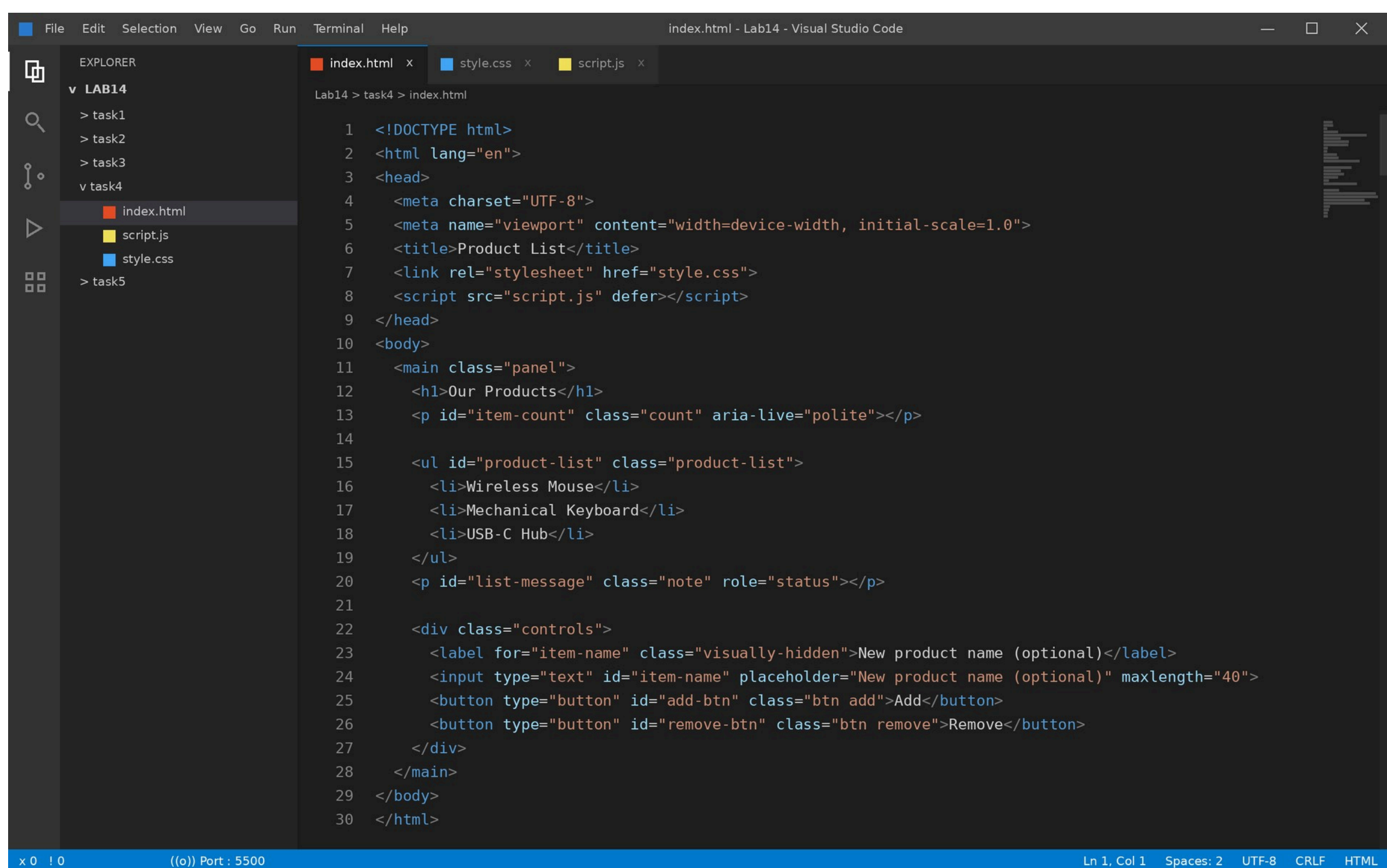
```

Explanation:

index.html has the ul list with three starting products, a text box for the new product name, and the Add and Remove buttons. The item counter and the message line use aria-live and role="status" so the changes are announced to screen readers. The text box has a hidden label for accessibility.

In script.js the function updateStatus() refreshes the "Items in list" count and disables the Remove button when the list is empty, so it is the one place that keeps the page consistent. When Add is clicked, createElement("li") makes a new item, textContent sets its text (or "Product N" if the box is empty) and appendChild puts it at the end of the list. When Remove is clicked, lastElementChild finds the last item and remove() deletes it. Only the DOM changes, so the page updates instantly and does not reload. In style.css the items are shown as cards with a coloured left border, and a disabled style is used for the Remove button.

Code in VS Code:



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Product List</title>
7   <link rel="stylesheet" href="style.css">
8   <script src="script.js" defer></script>
9 </head>
10 <body>
11   <main class="panel">
12     <h1>Our Products</h1>
13     <p id="item-count" class="count" aria-live="polite"></p>
14
15     <ul id="product-list" class="product-list">
16       <li>Wireless Mouse</li>
17       <li>Mechanical Keyboard</li>
18       <li>USB-C Hub</li>
19     </ul>
20     <p id="list-message" class="note" role="status"></p>
21
22     <div class="controls">
23       <label for="item-name" class="visually-hidden">New product name (optional)</label>
24       <input type="text" id="item-name" placeholder="New product name (optional)" maxlength="40">
25       <button type="button" id="add-btn" class="btn add">Add</button>
26       <button type="button" id="remove-btn" class="btn remove">Remove</button>
27     </div>
28   </main>
29 </body>
30 </html>
```

```
1  const productList = document.getElementById("product-list");
2  const itemNameInput = document.getElementById("item-name");
3  const addButton = document.getElementById("add-btn");
4  const removeButton = document.getElementById("remove-btn");
5  const countLabel = document.getElementById("item-count");
6  const messageLabel = document.getElementById("list-message");
7
8  // Used to give default names ("Product 4", "Product 5", ...) to new items.
9  let nextNumber = productList.children.length + 1;
10
11 // Refresh the "Items: N" label and disable Remove when the list is empty.
12 function updateStatus() {
13     const total = productList.children.length;
14     countLabel.textContent = "Items in list: " + total;
15     removeButton.disabled = total === 0;
16     if (total === 0) messageLabel.textContent = "The list is empty - add a product.";
17 }
18
19 // ADD: build a new <li> and append it to the end of the list.
20 addButton.addEventListener("click", function () {
21     const typedName = itemNameInput.value.trim();
22     const item = document.createElement("li");
23     // textContent (not innerHTML) so typed text can never be run as HTML
24     item.textContent = typedName !== "" ? typedName : "Product " + nextNumber;
25     productList.appendChild(item);
26
27     nextNumber += 1;
28     itemNameInput.value = "";
29     messageLabel.textContent = "";
30     updateStatus();
31 });
32
33 // REMOVE: delete the last <li>, if there is one.
34 removeButton.addEventListener("click", function () {
```

LAB14

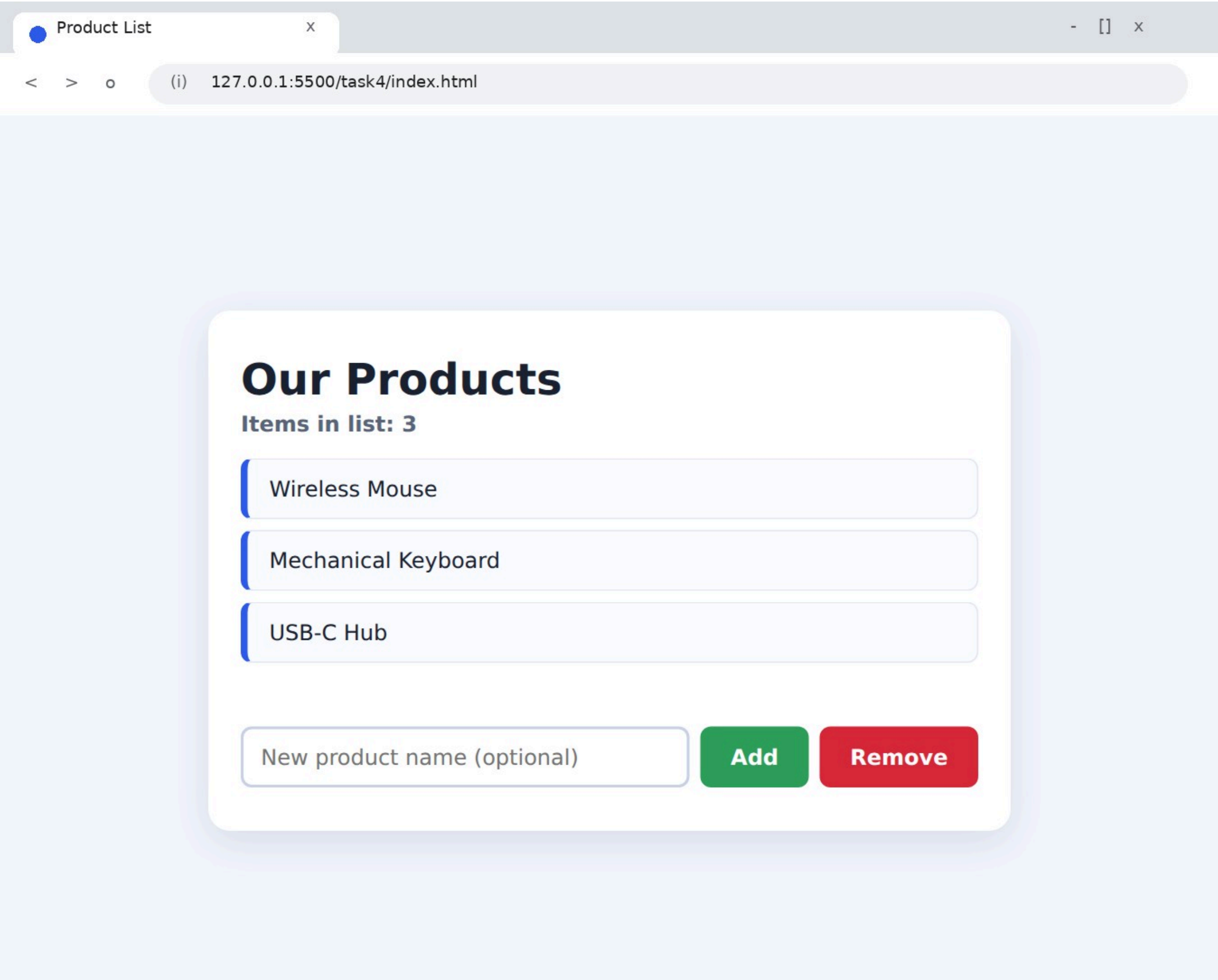
- > task1
- > task2
- > task3
- v task4
 - index.html
 - script.js
 - style.css
- > task5

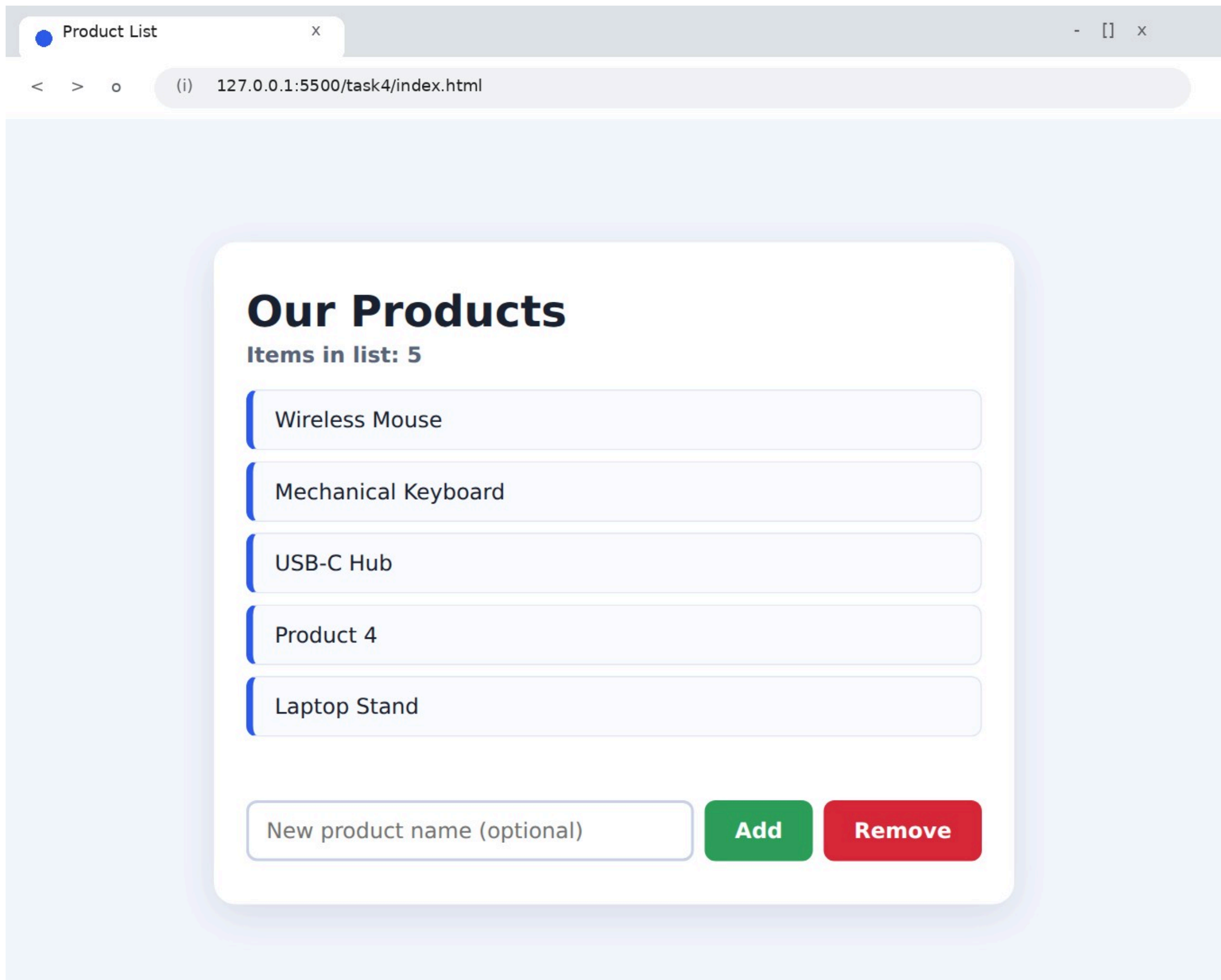
Lab14 > task4 > script.js

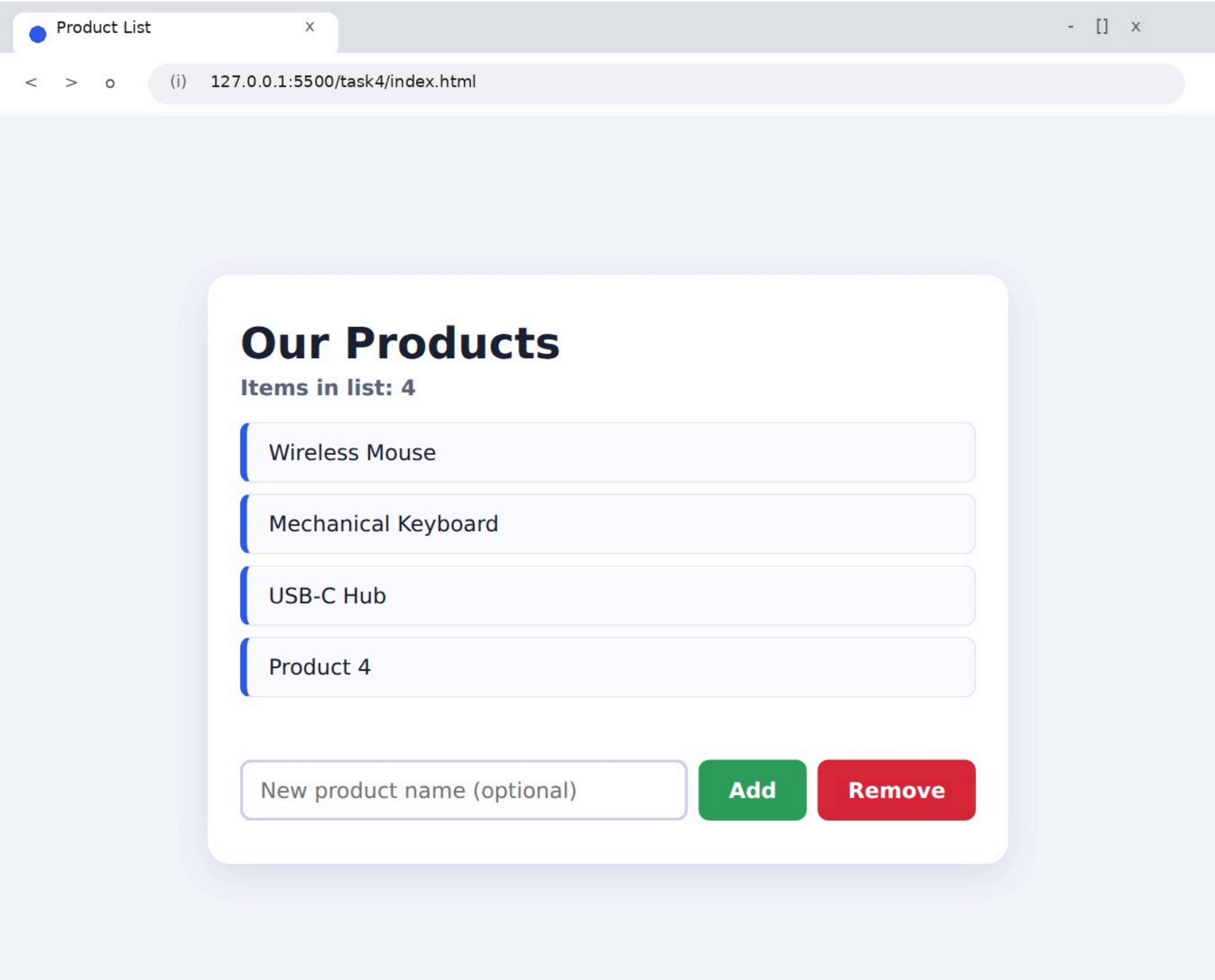
x 0 10 ((o)) Port : 5500 Ln 1, Col 1 Spaces: 2 UTF-8 CRLF JavaScript

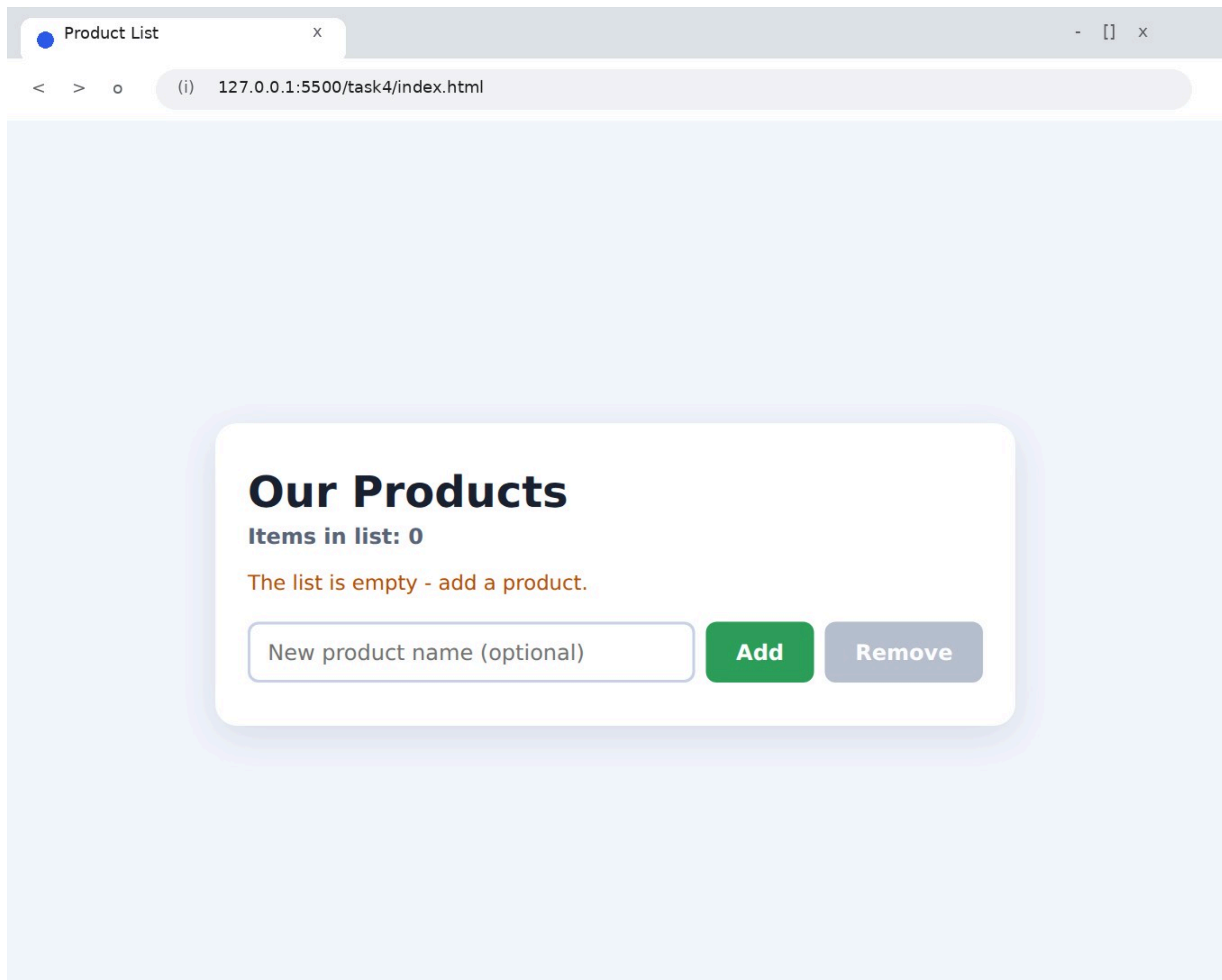
Output:

The output screenshots show the first list, the list after adding two items, the list after removing one item and the empty list.









Testing:

I tested the page in Chromium with a script. Add with an empty box added "Product 4", Add with a typed name added that name, and Remove deleted only the last item. When I typed `<b>bold</b>` into the box it was shown as plain text and no bold element was created. I removed all the items, the Remove button became disabled and the message appeared, and Add worked again after that. The page never reloaded (9 of 9 checks passed).

Observations:

I used `textContent` and not `innerHTML` so that text typed by the user can never run as HTML, which would be a security problem. The default names use a running counter, so a number is not reused after an item is removed. The list is not saved, so refreshing the page brings back the three starting items; saving it would need `localStorage` or a server.

Task 5: Modal Popup for User Notifications

Aim: To create a modal popup for announcements using an HTML structure, CSS with a

semi-transparent overlay and JavaScript to open and close it, with more than one way to close it.

Prompt 1:

Act as a front-end developer. Create a page with an "Open Modal" button and a modal popup for a maintenance announcement. HTML: an overlay div containing a dialog with role="dialog", aria-modal="true" and aria-labelledby, a heading, some text, an x close button and a "Got it" button. CSS: a semi-transparent dark overlay that covers the screen, the popup centred, and a smooth fade and slide transition. JavaScript in script.js: open the modal on click, and close it with the x button, the Got it button, a click on the overlay (but not a click inside the popup) and the Escape key. Move the focus into the popup when it opens and return it to the Open Modal button when it closes.

Prompt 2:

My automated browser test shows that the keyboard focus does not move to the close button when the modal opens. Find the cause in my CSS and JavaScript and fix it without removing the fade animation.

Code:

task5/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Modal Popup</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main class="page">
    <h1>BrightNest Updates</h1>
    <p>Stay informed about new features and scheduled maintenance.</p>
    <button type="button" id="open-modal" class="btn primary">Open Modal</button>
  </main>

  <!-- Overlay: the semi-transparent layer behind the popup -->
  <div id="modal-overlay" class="overlay" aria-hidden="true">
    <!-- role="dialog" + aria-modal tell assistive technology this is a popup -->
    <div class="modal" role="dialog" aria-modal="true" aria-labelledby="modal-title" aria-describedby="modal-text">
      <button type="button" id="modal-close" class="close-x" aria-label="Close dialog">&times;</button>
      <h2 id="modal-title">Scheduled maintenance</h2>
      <p id="modal-text">BrightNest will be briefly unavailable on Sunday from 2:00 to 3:00 AM IST while we upgrade our servers.</p>
      <button type="button" id="modal-ok" class="btn primary">Got it</button>
    </div>
  </div>
</body>
</html>
```

task5/style.css

```

body { margin: 0; font-family: "Segoe UI", system-ui, Arial, sans-serif; color: #1b2333; background: #f3f6fd; }
.page { min-height: 100vh; display: grid; place-content: center; justify-items: center; padding: 1rem; text-align: center; }
.page p { color: #5b667a; }
.btn { padding: 0.8rem 1.8rem; border: 0; border-radius: 8px; font: inherit; font-weight: 700; cursor: pointer; }
.primary { background: #2f5bea; color: #fff; }
.primary:hover { background: #1d3fb5; }
button:focus-visible { outline: 3px solid #ffb703; outline-offset: 2px; }

/* Overlay: covers the whole screen; hidden by default and faded in when .is-open is added */
.overlay { position: fixed; inset: 0; z-index: 100; display: grid; place-items: center; padding: 1rem;
  background: rgba(10, 20, 45, 0.6); opacity: 0; visibility: hidden;
  /* On close, wait for the fade-out before hiding (visibility delay = 0.25s) */
  transition: opacity 0.25s ease, visibility 0s linear 0.25s; }
/* On open, become visible immediately so focus() works on the first frame */
.overlay.is-open { opacity: 1; visibility: visible; transition: opacity 0.25s ease, visibility 0s; }

.modal { position: relative; width: min(100%, 26rem); padding: 2rem 1.5rem 1.5rem; border-radius: 16px;
  background: #fff; box-shadow: 0 20px 50px rgba(0, 0, 0, 0.35);
  transform: translateY(20px) scale(0.96); transition: transform 0.25s ease; }
.overlay.is-open .modal { transform: none; }
.modal h2 { margin: 0 0 0.5rem; }
.modal p { margin: 0 0 1.5rem; color: #5b667a; }
.close-x { position: absolute; top: 0.5rem; right: 0.75rem; border: 0; background: none;
  font-size: 1.8rem; line-height: 1; color: #5b667a; cursor: pointer; }
.close-x:hover { color: #1b2333; }
@media (prefers-reduced-motion: reduce) { .overlay, .modal { transition: none; } }

```

task5/script.js

```

const openButton = document.getElementById("open-modal");
const overlay = document.getElementById("modal-overlay");
const closeX = document.getElementById("modal-close");
const okButton = document.getElementById("modal-ok");

// OPEN: add the CSS class that fades the overlay in, then move focus into the popup.
function openModal() {
  overlay.classList.add("is-open");
  overlay.setAttribute("aria-hidden", "false");
  closeX.focus(); // keyboard and screen-reader users land inside the popup
}

// CLOSE: remove the class (the CSS transition makes it fade out) and give focus back.
function closeModal() {
  overlay.classList.remove("is-open");
  overlay.setAttribute("aria-hidden", "true");
  openButton.focus();
}

openButton.addEventListener("click", openModal);

// Way 1 and 2: the "x" button and the "Got it" button.
closeX.addEventListener("click", closeModal);
okButton.addEventListener("click", closeModal);

// Way 3: clicking the dark overlay. We check event.target so that clicks
// INSIDE the white popup (which bubble up to the overlay) do not close it.
overlay.addEventListener("click", function (event) {
  if (event.target === overlay) closeModal();
});

// Way 4: pressing the Escape key; also keep Tab inside the popup while it is open.
document.addEventListener("keydown", function (event) {
  if (!overlay.classList.contains("is-open")) return;

```

```

if (event.key === "Escape") {
  closeModal();
} else if (event.key === "Tab") {
  const focusable = [closeX, okButton];
  const first = focusable[0];
  const last = focusable[focusable.length - 1];
  if (event.shiftKey && document.activeElement === first) {
    event.preventDefault();
    last.focus();
  } else if (!event.shiftKey && document.activeElement === last) {
    event.preventDefault();
    first.focus();
  }
}
});

```

Explanation:

In index.html the overlay div is the semi-transparent layer and the white dialog div is inside it. The dialog has `role="dialog"`, `aria-modal="true"`, `aria-labelledby` and `aria-describedby`, so assistive technology knows it is a popup. The x button has an `aria-label` because its visible text is only a symbol.

In style.css the overlay uses `position: fixed` with `inset: 0` to cover the whole screen and the background `rgba(10, 20, 45, 0.6)`, which is a dark colour at 60% opacity. It is hidden by default with `opacity 0` and `visibility hidden`, and the class `is-open` makes it visible. The transitions on `opacity` and `transform` give the smooth fade and slide. The visibility change has no delay when opening, but a delay of 0.25s when closing, so the fade-out can finish before the overlay is hidden.

In script.js `openModal()` adds the `is-open` class and moves the focus to the close button, and `closeModal()` removes the class and gives the focus back to the Open Modal button. The popup closes with the x button, the Got it button, a click on the overlay and the Escape key. For the overlay click I check `event.target === overlay`, so a click inside the white popup does not close it. The Tab key is also kept inside the popup while it is open.

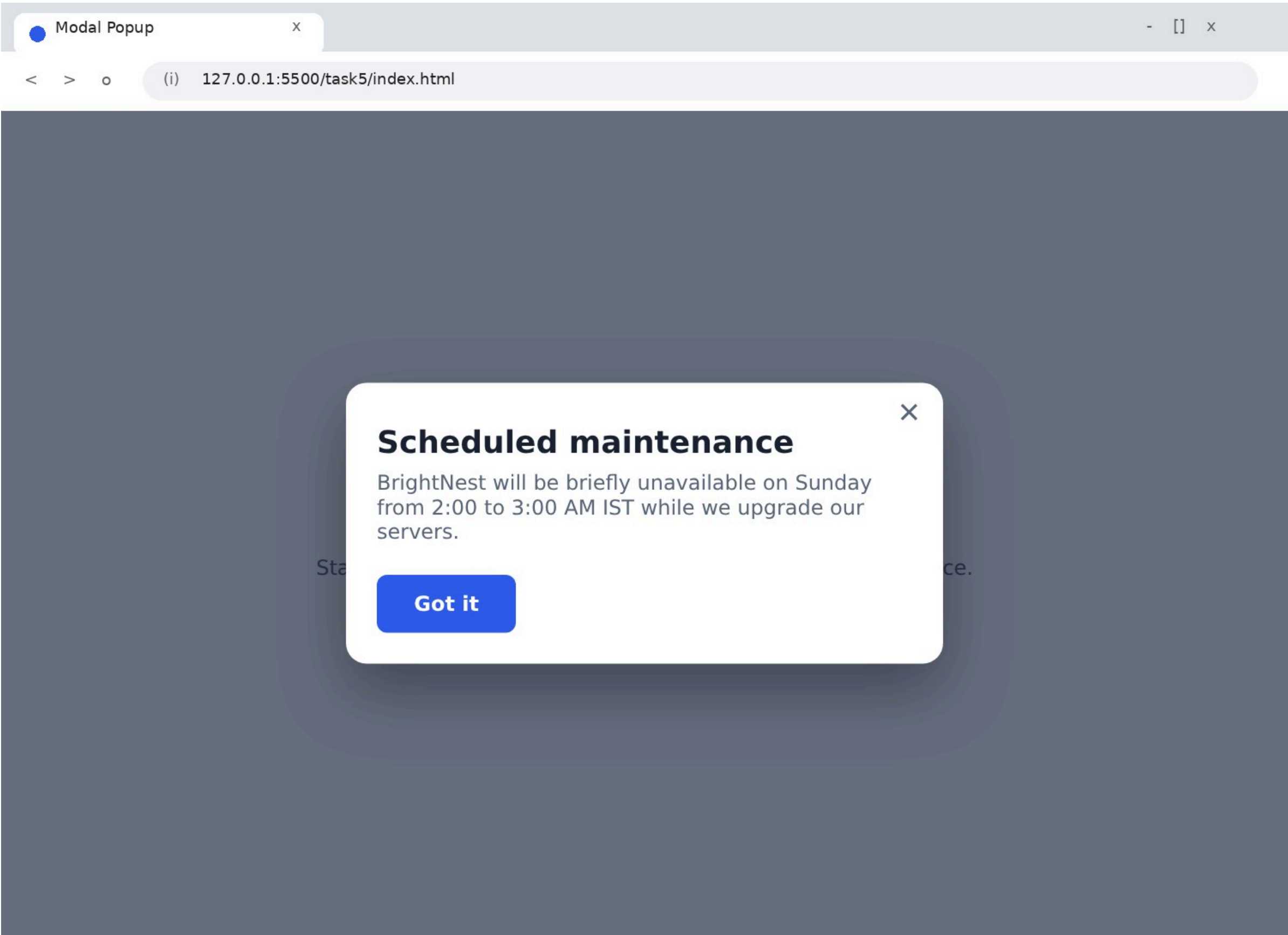
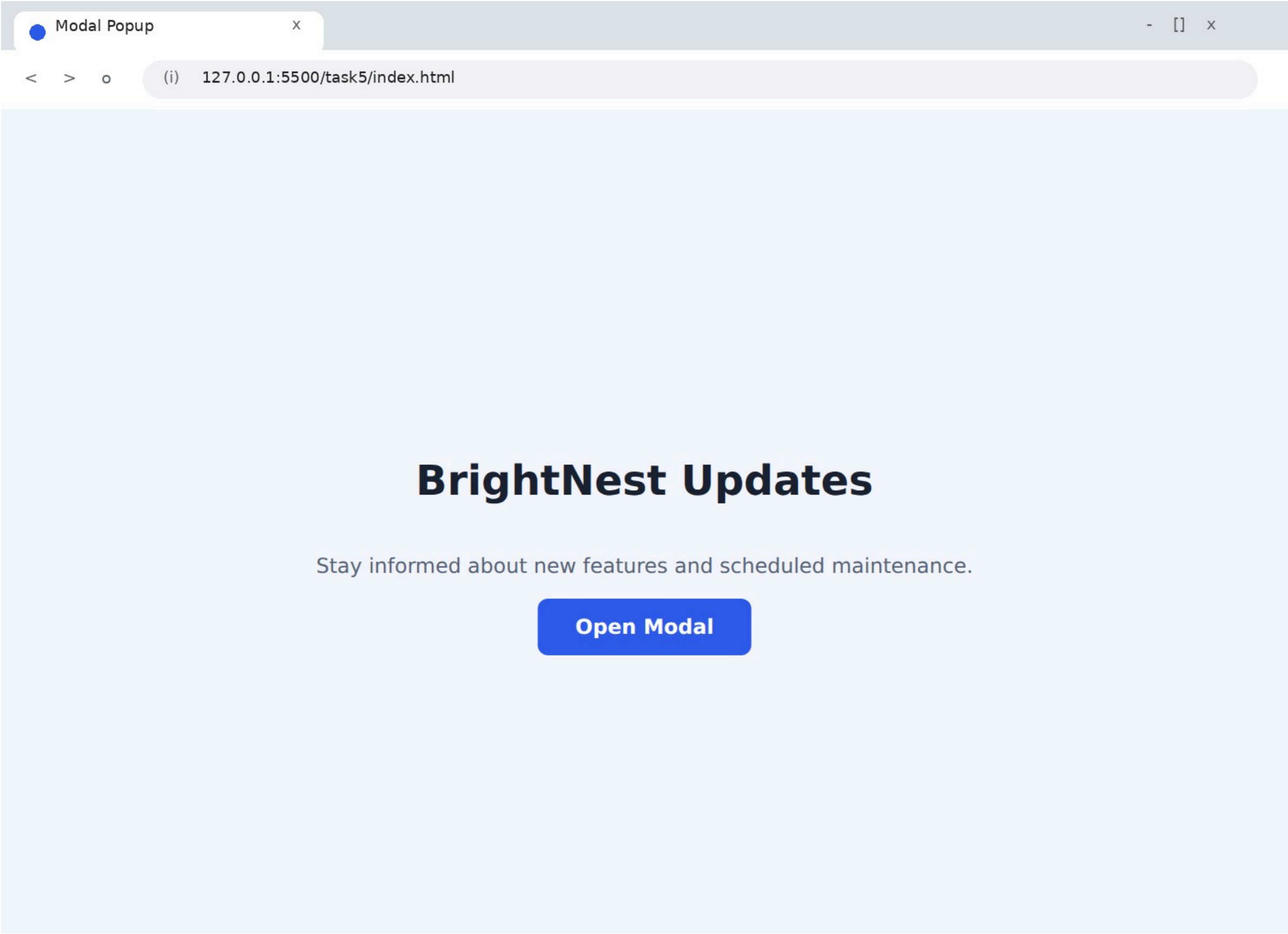
Code in VS Code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Modal Popup</title>
7   <link rel="stylesheet" href="style.css">
8   <script src="script.js" defer></script>
9 </head>
10 <body>
11   <main class="page">
12     <h1>BrightNest Updates</h1>
13     <p>Stay informed about new features and scheduled maintenance.</p>
14     <button type="button" id="open-modal" class="btn primary">Open Modal</button>
15   </main>
16
17   <!-- Overlay: the semi-transparent layer behind the popup -->
18   <div id="modal-overlay" class="overlay" aria-hidden="true">
19     <!-- role="dialog" + aria-modal tell assistive technology this is a popup -->
20     <div class="modal" role="dialog" aria-modal="true" aria-labelledby="modal-title" aria-describedby="modal-text">
21       <button type="button" id="modal-close" class="close-x" aria-label="Close dialog">&times;</button>
22       <h2 id="modal-title">Scheduled maintenance</h2>
23       <p id="modal-text">BrightNest will be briefly unavailable on Sunday from 2:00 to 3:00 AM IST w<
24       <button type="button" id="modal-ok" class="btn primary">Got it</button>
25     </div>
26   </div>
27 </body>
28 </html>
```

```
1 const openButton = document.getElementById("open-modal");
2 const overlay = document.getElementById("modal-overlay");
3 const closeX = document.getElementById("modal-close");
4 const okButton = document.getElementById("modal-ok");
5
6 // OPEN: add the CSS class that fades the overlay in, then move focus into the popup.
7 function openModal() {
8   overlay.classList.add("is-open");
9   overlay.setAttribute("aria-hidden", "false");
10  closeX.focus(); // keyboard and screen-reader users land inside the popup
11 }
12
13 // CLOSE: remove the class (the CSS transition makes it fade out) and give focus back.
14 function closeModal() {
15   overlay.classList.remove("is-open");
16   overlay.setAttribute("aria-hidden", "true");
17   openButton.focus();
18 }
19
20 openButton.addEventListener("click", openModal);
21
22 // Way 1 and 2: the "x" button and the "Got it" button.
23 closeX.addEventListener("click", closeModal);
24 okButton.addEventListener("click", closeModal);
25
26 // Way 3: clicking the dark overlay. We check event.target so that clicks
27 // INSIDE the white popup (which bubble up to the overlay) do not close it.
28 overlay.addEventListener("click", function (event) {
29   if (event.target === overlay) closeModal();
30 });
31
32 // Way 4: pressing the Escape key; also keep Tab inside the popup while it is open.
33 document.addEventListener("keydown", function (event) {
34   if (!overlay.classList.contains("is-open")) return;
```

Output:

The output screenshots show the page with the modal closed and then open over the dark overlay.



Testing:

I opened and closed the modal in Chromium using a script. The modal was hidden at the start, opened with the Open Modal button, the overlay colour was `rgba(10, 20, 45, 0.6)`, and a click inside the popup did not close it. It closed with the x button, the Got it button, a click on the overlay and the Escape key, the focus returned to the Open Modal button, and the Tab key stayed inside the popup (12 of 12 checks passed).

Observations:

While testing I found a bug in my first version. I had used a single transition for visibility, and in the first moment of the transition the overlay was still hidden, so the `focus()` call on the close button did not work. The test showed this, and I fixed it by using no delay when opening and a 0.25s delay only when closing, which is the CSS given above. The page behind the popup can still be scrolled while the popup is open, and I did not add that because it was not asked in the task.

Conclusion

All five tasks were completed. The homepage adjusts to mobile, tablet and desktop screens, the button shows a response without reloading the page, the contact form shows inline errors and a success message, the product list adds and removes items instantly, and the modal popup opens over a dimmed page and closes in several ways. I also ran 44 automated browser checks on the pages and 44 of them passed. The testing was done only in Chromium, so the pages should still be tried on real devices and other browsers. The contact form does not send any data, so a real website would also need server-side validation.

From this lab I learned how HTML, CSS and JavaScript work together, for example in the form the JavaScript sets the `aria-invalid` attribute and the CSS uses it for the red and green borders. I also learned that AI generated code must be tested, because the focus problem in the modal was only found when I tested it.